MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

PORTABLE PERSONAL ENVIRONMENT

LICENSE THESIS

Graduate: **George TOFAN**

Supervisor: **Conf. Dr. Ing. Ciprian OPRISA**

2026

Contents

Chapter 1 Introduction - Project Context **1**

Chapter 2 Project Objectives **3**
2.1 Purpose and Core Properties . . . 3
2.2 Functional Requirements . . . 3
2.3 Non-Functional Requirements . . . 4
2.3.1 Security . . . 4
2.3.2 Performance . . . 4
2.3.3 Reliability and Robustness . . . 5
2.3.4 Ease of Use . . . 5
2.4 Scope Boundaries . . . 5
2.5 The Path Toward the Objectives . . . 5
2.6 Main Challenges . . . 5

Chapter 3 Bibliographic Research **6**
3.1 State-of-the-Art Competing Approaches . . . 6
3.1.1 Live USB . . . 6
3.1.2 Cloud Virtual Machines . . . 7
3.2 Kernel Namespaces . . . 7
3.3 Containerization Mechanisms . . . 8
3.3.1 Application Containers . . . 8
3.3.2 System Containers . . . 10
3.4 Best Suited Technology . . . 11
3.5 Closest Current Alternative . . . 11

Chapter 4 Analysis and Theoretical Foundation **13**
4.1 General Architecture . . . 13
4.1.1 Conceptual Architecture . . . 13
4.1.2 External Drive Structure . . . 14
4.1.3 Host Computer Structure . . . 15
4.2 The Host Configuration Tool . . . 16
4.2.1 Initial Checks . . . 16
4.2.2 Configuration Flow . . . 16
4.2.3 Removal Flow . . . 17
4.2.4 Idempotency and Robustness . . . 17
4.3 The Session Launcher . . . 18
4.3.1 Startup Phases . . . 18
4.3.2 Environment Creation . . . 18
4.3.3 Host-Container Session Bridging . . . 19
4.3.4 Personal User Configuration Inside the Container . . . 20
4.3.5 Main User Interface Structure . . . 20
4.3.6 Launcher Structure . . . 20
4.3.7 Session Termination . . . 21
4.3.8 The Unmount Mechanism . . . 22
4.4 Integration in the Linux Filesystem . . . 22

4.5 Security Analysis . . . 24
4.5.1 Tampering with the Mounting Rules . . . 24
4.5.2 A Stolen Drive . . . 24
4.5.3 In-Container Compromise Escalating to the Host . . . 25
4.5.4 A Compromised Host Tampering with the Container . . . 25
4.5.5 A Malicious Launcher . . . 25
4.5.6 Human Error and Unattended Sessions . . . 26

Chapter 5 Detailed Design and Implementation 27
5.1 Design-Level Architecture Overview . . . 27
5.2 Use Cases . . . 28
5.2.1 Actors . . . 28
5.2.2 Use Case Inventory . . . 29
5.2.3 Central Usage Scenario . . . 30
5.3 Drive Formatting . . . 31
5.4 The Configuration Tool . . . 32
5.4.1 Technology . . . 32
5.4.2 Data Modeling . . . 33
5.4.3 The CLI and Orchestration Layer . . . 34
5.4.4 Implementation Details . . . 35
5.4.5 The Templates Module . . . 37
5.5 The Session Launcher . . . 37
5.5.1 Technology . . . 38
5.5.2 Module Decomposition . . . 38
5.5.3 Managed State and the Singleton Pattern . . . 38
5.5.4 Host Tool Abstraction . . . 39
5.5.5 The Centralized Container Execution Layer . . . 41
5.5.6 Live Application Discovery . . . 41
5.5.7 The File Transfer Directory . . . 42
5.5.8 AppImage Environment Cleanup . . . 42
5.5.9 The Session Environment File . . . 43
5.5.10 The D-Bus Session Bus and Persistent Settings . . . 43
5.6 The Launcher Frontend . . . 44
5.6.1 The Command Bridge and the Uniform Result Contract . . . 44
5.6.2 The View State Machine . . . 45
5.6.3 Modeling the Launcher Entries . . . 46

Chapter 6 Testing and Validation 48
6.1 Self-Validation Through Re-Execution . . . 48
6.2 Automated Runtime Tests . . . 50
6.2.1 Implicit Host Integrity Checks Before Boot . . . 50
6.2.2 Runtime Integrity Diagnostics . . . 50
6.3 Heterogeneous Device Compatibility . . . 51
6.4 Performance Validation . . . 53
6.4.1 Processor Performance . . . 53
6.4.2 Graphics Performance and GPU Passthrough . . . 54
6.4.3 System Call Latency . . . 54
6.4.4 Application Launch and Execution Time . . . 55

6.4.5 Impact of the Connection Protocol 56
6.5 Usability Validation 57
6.6 Security Validation 57

Chapter 7 User's Manual **61**
7.1 Prerequisites 61
7.2 Building the Two Components 61
7.3 Preparing the Drive 61
7.4 Host Configuration 62
7.4.1 Configuring a Host 62
7.4.2 Removing the Configuration 63
7.4.3 Idempotency and Help 63
7.5 Using the Environment 63
7.5.1 Unlocking the Drive and Opening the Launcher 63
7.5.2 First Run Configuration 63
7.5.3 Everyday Use 64
7.5.4 Shutting Down 64

Chapter 8 Conclusions **67**
8.1 Summary of Contributions 67
8.2 Critical Analysis of the Results 67
8.3 Possible Improvements and Further Development 68

Bibliography **69**

Appendix A Full Performance Measurements **71**

Chapter 1. Introduction - Project Context

Laptops and desktop computers have become essential across the educational field, from early schooling up to university studies, in professional work, and in personal life, where they are often needed to break free from the limitations of mobile devices. In most of these settings, an essential aspect is portability: students move from one classroom to another and continue their work at home, careers increasingly offer and require hybrid and flexible work environments, and individuals often need or own multiple devices specialized for different purposes, such as a gaming desktop computer alongside a lightweight laptop.

Especially for students in STEM, the need for a personal environment in each classroom and at home becomes obvious, as much of their work is tied to complex industry-standard programs that are essential in both engineering and research. As a result, the use of personal laptops in classrooms has become almost mandatory, while the computers provided by the institution remain almost entirely unused, for the following reasons:

- *Lack of security* — with all files and configurations being public, personal projects and data are accessible to all other students, potentially leading to data being stolen or permanently deleted. This makes the provided computers a very unreliable storage medium: after each session, all accounts and sensitive data must be thoroughly cleaned to avoid leaks, and all important work and progress must be backed up.
- *Difficult work file transfer* — to avoid the security risks above, to continue working on projects and exercises between classes or to keep previous materials as reference, students have to go through tedious and frequent manual transfer and management before and after each class, using difficult to configure cloud file management or physical storage media (which are often disabled or introduce security concerns of their own).
- *Difficult installations and configurations* — most of the prerequisite configurations required for practical classes must be done by the system administrator. In case of human or other errors, the user usually does not have the privileges needed to fix the configuration or install the missing packages in time. Furthermore, most custom configurations are not only discouraged, but impossible, due to the same lack of privileged access, limiting what would otherwise be great learning opportunities.

Although device portability, especially in the area of laptops, has seen great progress in the context of an increasingly dynamic lifestyle, three main limitations still prevail:

- truly compact and performant laptops demand a premium price point;
- as compact as they may get, the laptop itself and its charger still require a backpack or a serious accessory for transportation;
- there are physical hardware limitations to what a laptop can do, especially regarding power consumption and heat dissipation, cases that require using a separate desktop computer, which in turn raises further problems of file synchronization and double configuration and maintenance.

Therefore, student convenience is mostly limited by budget: either affording a

compact and performant device, or being bound to carrying a heavy laptop and charger, or, even worse, having to deal with the severe limitations of the classroom computers. This leaves considerable room for improvement, both in leveraging the classroom hardware that already exists and in providing a better experience to all students, those who already own performant and compact, but still flawed, laptops, as well as those struggling with the budget.

Common workarounds adopted by both students and institutions also come with considerable performance and ease-of-use disadvantages, such as the live USB method, which boots a personal operating system from an external drive on the classroom computer [1], and the cloud personal VM accessed from the classroom computer [2].

The purpose of this project is to propose and describe the Portable Personal Environment (PPE), a viable solution to the classroom computing portability issue, allowing students to keep all of their data, applications and configurations on a compact external drive that is compatible and seamlessly integrated with both the university computers and their own home environment. The workflow is simply plugging the drive into any pre-configured computer and continuing to work on all personal files and programs in a secure and customizable environment, while leveraging the computing power of the chosen host device.

Such an architecture is achievable by building on the already existing infrastructure and mechanisms of OS-level virtualization and containerization, intersecting them with the goal of portable personal computing to form a cross-device, cohesive system.

This approach bridges the gap between the personalization and portability advantages of the laptop and the already-paid-for capabilities of the classroom computers, while bypassing the described limitations of each. Even more, it offloads configuration duties from the classroom administrators to the students, facilitating learning opportunities for the students accommodating with their new tools and environment, while easing the administrators' work.

The remainder of this thesis follows the path taken in building the PPE. Chapter 2 formulates the objectives of the project, together with the functional and non-functional requirements the solution must meet. Chapter 3 studies existing methods of achieving portability and weighs them against these goals. Chapter 4 lays out the theoretical foundation and the conceptual solution, while Chapter 5 details the design and implementation of its two components: the host configuration tool and the session launcher. Finally, Chapter 6 evaluates the viability of the resulting system, Chapter 7 provides a user's manual, and Chapter 8 draws the conclusions.

Chapter 2. Project Objectives

2.1. Purpose and Core Properties

The purpose of the PPE is to provide a single, self-contained personal computing environment that exists entirely on portable storage and executes natively on any compatible and preconfigured host. The prerequisite host configuration should be minimal and allow the environment to reuse the computer's processing, operating system and peripheral resources, while remaining mostly isolated and operable without administrative privileges.

Concretely, the environment must meet the following properties:

- *Portability / host agnosticism* — the same environment runs unchanged across heterogeneous machines;
- *Persistence and self-containment* — all data, applications and configuration live on the drive and survive across sessions and across different hosts;
- *Native execution* — computation, including hardware acceleration, runs on the host computer's own hardware;
- *Isolation* — the environment is protected from the host, so that user data stays confidential and intact even on a shared and untrusted machine;
- *Unprivileged operation* — the user works without administrative rights on the host, with the privileged setup kept completely separate from normal use.

These specifications decompose into the functional and non-functional objectives presented in the following sections. Among them, the principal ones are treated in greater detail throughout the following sections: the performance objectives (led by native execution on the host's own hardware), unprivileged operation, the running of GUI applications, and the isolation and protection of files.

2.2. Functional Requirements

The functional requirements describe what the environment lets the user do. For each, its meaning and its role in the overall solution are detailed below:

- *Running of GUI applications* — the environment provides a graphical launcher interface that presents the container's applications and automatically keeps this list up to date as programs are installed or removed. Alongside it, the container terminal remains available for configuration work and for running CLI applications. This is the primary way students interact with the programs their work depends on.
- *Privileged access inside the environment* — inside the environment the user has true root capabilities, free to install, configure and modify the system as needed. This directly answers the lack of privileges that limits configuration and learning on

the institutional computers, and is made safe for the host through the unprivileged host operation described below.
- *Package management inside the container* — the user installs and manages software through the container distribution's own native package manager, seamlessly and without depending on a system administrator.
- *File transfer between the environment and the host* — files can be moved seamlessly in both directions between the container and the host.
- *Host configuration* — a dedicated tool prepares a host for running the environment, with support for varied host distributions and hardware, and a cleanup step that removes an existing configuration so it can later be updated. Broad host support is what makes the same drive usable across the heterogeneous machines found in different classrooms, as well as on personal devices.
- *Native network connectivity* — the environment has direct network access on the host.
- *Environment configuration* — an interactive utility lets the user configure their own distribution and identity inside the environment, and the system supports a variety of container distributions. This keeps the environment personal and customizable.

2.3. Non-Functional Requirements

The non-functional requirements describe the qualities the environment must exhibit while providing the functionality above, being grouped into different categories.

2.3.1. Security

- *Unprivileged operation* — although the user acts with full privileges inside the environment, the same privileged instructions are translated into unprivileged ones on the host, by mapping the in-environment privileges onto an unprivileged host identity, a mechanism referred to as UID translation. The privileged setup is kept entirely separate from normal use, so that day-to-day work never requires administrative rights on the host.
- *Isolated and protected files* — the environment is protected from host-side modification, appearing read-only from the host, so that a malicious or compromised host cannot tamper with or compromise the container.
- *Complete drive encryption* — the entire drive is encrypted, keeping the stored data confidential even if the drive is lost, stolen or accessed on an untrusted machine.

2.3.2. Performance

- *Native GPU acceleration* — graphics-intensive applications run with hardware acceleration on the host's own GPU;
- *Offloading of heavy OS components to the host* — the heaviest parts of the operating system, the kernel and the display server, live and run on the host;
- *Close-to-native read/write speeds* — storage access stays close to native speeds, so that the drive-based environment does not become a bottleneck;
- *Minimal package installation sizes* — installed packages stay small, keeping the drive from filling up quickly.

2.3.3. Reliability and Robustness

The system implements mechanisms that make sure it is never left in any inconsistent state, as well as others that check for and signal misconfigurations.

2.3.4. Ease of Use

All of the functional features are seamlessly and intuitively integrated, with accessibility considerations kept in mind throughout.

2.4. Scope Boundaries

To keep the project focused, several aspects are deliberately left outside its scope. The PPE does not cover:

- automatic formatting of the external drive;
- mitigating all discovered security vulnerabilities;
- providing more package management assistance than the container distribution's default terminal and package manager;
- supporting all host distributions — only Ubuntu/Debian have been thoroughly tested, although the design is highly extensible toward integrating others;
- support for discrete GPUs, due to the incompatible driver configurations between models;
- support for the X11 display server on the host.

2.5. The Path Toward the Objectives

The objectives are pursued along a path that also shapes the remainder of this thesis. It begins by *studying* existing methods of achieving portability, then moves on to *designing* a solution that meets the requirements, from its high-level architecture down to its individual main components. The proposed solution is then *evaluated* against its theoretical security concerns, *developed* into a working system, and finally *tested* in order to evaluate its viability.

2.6. Main Challenges

Reaching these objectives involves a number of challenges that recur throughout the process:

- bridging the gap between the privileged and unprivileged spaces;
- supporting heterogeneous hosts with as few assumptions as possible;
- dealing with self-referential mounting and unmounting;
- testing, which is difficult to automate and slow, being bound to heterogeneous hardware.

Chapter 3. Bibliographic Research

3.1. State-of-the-Art Competing Approaches

Both of the approaches discussed below attempt the PPE's goal, a personal environment usable on arbitrary host hardware, but they sit at opposite poles of a single axis: *where execution happens*. The live USB executes locally, but only by completely replacing the host OS. The cloud VM executes remotely, reducing the host to a terminal. Neither coexists with the host while using its hardware natively.

3.1.1. Live USB

The live USB mechanism boots a full personal operating system directly from the external drive. The host is reduced to bare hardware, with its installed OS being bypassed entirely. There are generally two modes: ephemeral, resetting on each boot, and persistent, keeping a writable overlay on the drive that stores changes. The persistent mode is the direct point of comparison for the PPE. In both cases, using it requires rebooting the host and selecting the USB drive in the firmware boot order.

This approach shares the PPE's central strength: native execution on the host, taking advantage of its CPU, GPU and RAM without any virtualization overhead, and with direct hardware access that needs no extra passthrough work. It is also self-contained and portable, not interacting at all with the host's installed system.

On the downside, by taking over the whole machine it needs a reboot and access to the firmware boot order. This represents both a security risk on classroom hardware, as it requires disabling *Secure Boot* and allowing the risky USB boot, and a performance drawback, since an entire operating system must boot before the system can be used. Running on bare hardware also provides no isolation from the host's persistent disks, allowing malicious users to corrupt or delete the host OS and its configurations completely.

There is also the issue of brittleness: booting on unknown and heterogeneous hardware often requires proprietary GPU and Wi-Fi drivers that are difficult to find and install and, more importantly, frequently incompatible between different hosts. Here the PPE's approach completely bypasses the limitation, reusing the host's already installed and working kernel and drivers.

Persistence, moreover, is an afterthought rather than a native feature: the core system, including the kernel, is read-only, with only a writable overlay on top. An improper shutdown can permanently corrupt this persistence file, and the kernel itself cannot be updated, nor can kernel-level drivers be installed for it [1]. As a further performance drawback, the whole OS is bottlenecked by the drive and its slower connection protocol (especially on USB 2.0 [3] and on flash media generally, due to poor random I/O [4]) which cripples everything from boot time to all subsequent operations. While the PPE is also limited by the case of a slow external storage or connection, once programs are

loaded in memory, they run natively, highly limiting the impact of transfer speeds.

3.1.2. Cloud Virtual Machines

A cloud VM is a persistent personal virtual machine hosted remotely. The user connects from the classroom machine through a remote-desktop client, while the environment lives and executes in the data center, the host being used only for its display and input.

Such remote execution setups are already used in academic practice. The University of Washington's College of Engineering, for instance, provides enrolled students with full virtual desktops accessible over the network from their own devices, including GPU-backed sessions for compute-intensive coursework [5].

One of the major advantages of this approach is its extreme host agnosticism, needing only a client and a network connection, with no pre-configuration, host privileges or Linux environment required on the host. The other is that it is fully persistent, centrally managed and strongly isolated from the host, providing excellent security together with flexible resource scaling.

A first disadvantage is that it does not use the host's computing power, which is the exact opposite of the PPE's purpose of leveraging already paid for local hardware. Integrating cloud VMs for students would require serious additional cloud infrastructure investment, and, beyond that initial cost, the recurring expense of hosting those desktops: licensing, cloud resources and the expertise to manage them. In the end, this approach only shifts rather than removing the budget burden that Chapter 1 frames.

It is also network-bound, with performance depending heavily on connection reliability, speed, latency and stability, all of which strongly affect responsiveness [2]. For the same reason, cloud VMs are unusable offline and especially weak for GUI and graphics-intensive work.

Each route therefore sacrifices something essential: the live USB gives up coexistence and reliable cross-host hardware support, while also introducing considerable security risks, whereas the cloud VM gives up local performance, offline use and one-time cost. The PPE targets the gap between them: native local execution like the live USB, but reusing the host's kernel and drivers, thereby avoiding the most severe bottlenecks, the need for a reboot and the cross-platform driver brittleness. On top of this, it provides persistence and isolation by design, and suffers from neither network dependence nor recurring costs.

3.2. Kernel Namespaces

Kernel namespaces are a mechanism integrated directly into the Linux kernel that provides the low-level means of isolating the different aspects of a process from the rest of the system. They are the building blocks of Linux containerization.

Alongside them sit control groups (cgroups), the complementary kernel mechanism responsible for managing and limiting the resource usage of groups of processes. Although cgroups are not relevant to the needs of this project, as the PPE should use the unrestricted hardware of the host, together with namespaces they form the basis of every containerization method, from Kubernetes pods and systemd's management of background services to, most notably, Docker.

In the context of this project, it would be possible to build the environment directly on top of these namespaces primitives, which would offer a great deal of flexibility. The drawback, however, is that doing so is highly laborious, which motivates the interest in relying instead on the existing implementations and abstractions built over them, examined in the following section.

3.3. Containerization Mechanisms

Building on the kernel namespaces described above, a range of containerization mechanisms add successive layers of abstraction over these low-level primitives, from tools that stay close to the raw namespaces to fully managed, high-level interfaces. From the namespace primitives, they branch into two families: application containers and system containers, as illustrated in Figure 3.1.

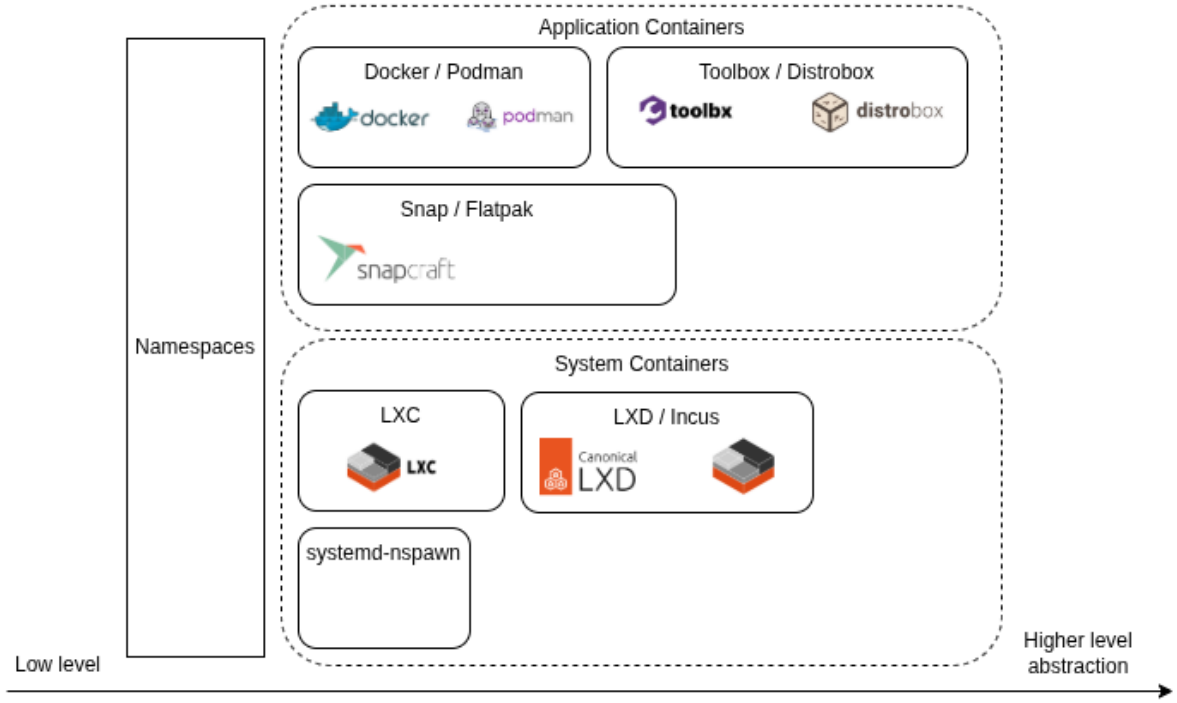


Figure 3.1: Containerization mechanisms, from the low-level kernel namespaces to increasingly high-level tools, split into the application container and system container branches.

3.3.1. Application Containers

Application containers package and isolate individual programs rather than entire systems. They rely on the same namespaces and control groups, but apply them toward the narrow goal of taking a single application, bundling it with its dependencies and running it in a confined sandbox that interacts with the host as little as possible. They are mostly a means of managing an application or service, not a persistent machine.

The mechanisms in the first two of the categories below share the benefits of leveraging bundling for cross-platform compatibility, as well as providing unprivileged operation. However, they are single application scoped and are designed to keep their persistent state tightly linked to the host.

Docker / Podman

Docker and Podman popularized the modern container model. They bundle an application together with a stripped-down user space to build an immutable image, a running container being a disposable instance of that image. Docker does not rely on any shared runtime from the host, bundling a minimal OS layer inside the image itself; it is targeted at reproducible, headless, server-side workloads, where containers are stateless and all persistent information must be explicitly declared in volumes. Podman is a daemonless, rootless reimplementation of the same interface as Docker. Docker itself was originally built on LXC, before replacing it with its own runtime [6].

In the context of this project, the image/volume model not only abstracts away the required low-level control, but also enforces a stateless design principle, which makes it difficult to work against when persisting an entire environment.

Snap / Flatpak

Snap and Flatpak build on the same primitives but orient them toward desktop use. Their purpose is application distribution and sandboxing: shipping a graphical program together with its libraries for cross-distribution compatibility, while confining what it can reach on the host.

In this context, the portal and shared-runtime sandbox model provides no real root filesystem, but only specific locations for accessing a shared runtime and storing settings, with portals configured to grant further access to the host. Even that persistence access is deeply coupled to the host. The sandbox, which is their most important mechanism, therefore heavily obstructs any system-wide use.

Toolbox / Distrobox

At first glance, Toolbox and Distrobox are the closest to the intentions of the PPE. They are wrappers over Podman and Docker that create containers meant to be kept customized and lived in ("pet containers") as opposed to the disposable use case their underlying mechanism was designed for. They were built for host integration, mounting directly into the home directory and providing deep integration without needing configuration. Toolbox is proprietary to Fedora/RHEL, while Distrobox is a general rewrite that provides wide compatibility.

In this context, their deepest and effortless host integration, GPU acceleration, sound, graphical libraries and networking passthrough all configured by default, is real, but it is achieved precisely through the tightly coupled host integration that the PPE is trying to avoid, and would require serious, deep customization to fight against. They also abstract away low-level control that this project needs, over the mount layout, UID mapping and read-only filesystem.

Overall, the application container category is unsuited to the PPE's use case, as it would mean working against several fundamental principles: Docker's intended statelessness, Snap and Flatpak's sandbox that obstructs system-wide use, and Distrobox's deep host integration and overly high level of abstraction.

3.3.2. System Containers

System containers virtualize an entire OS user space rather than a single application. They present themselves as full machines, with their own init system as PID 1, their own filesystem and their own services, while still sharing the host kernel. Over completely isolated VMs, they have the advantage of needing no hypervisor and no guest kernel, using only a fraction of the RAM and CPU cost, while the applications they run still perceive a full machine.

Also, unlike application containers, they are by design persistent, multi-application, complete environments, directly matching the needs of the PPE instead of forcing it to work against the basic principles of the technology.

The main limitation, however, is this category's historical target of headless, server use. GUI rendering and hardware passthrough therefore need explicit configuration work to reach the host's graphics drivers, display server socket (where security becomes an issue for X11, which exposes the whole host GUI, as opposed to Wayland), GPU devices and peripherals. With this, the remaining design problem shifts from fighting statelessness or too deep a host coupling toward how each tool handles GUI and hardware integration while preserving cross-host compatibility.

Each adding an extra layer of abstraction over the basic namespace tools, the most relevant openly available mechanisms in this category are systemd-nspawn, LXC and Incus.

systemd-nspawn

systemd-nspawn is built into systemd and designed for lightweight namespace containers. It is already present on the majority of Linux hosts and offers native, streamlined hardware and GUI socket passthrough.

While it is ubiquitous on Linux hosts and facilitates streamlined integration, it requires root access on the host. This is a dealbreaker, directly violating the essential unprivileged operation objective.

LXC

LXC is a low-level set of tools and libraries for system containers, where everything is managed manually, primarily through declarative configuration files. It is widely used, especially for production system containers, and is the foundation of tools such as Proxmox's backend and LXD/Incus, as well as, as mentioned earlier, the original foundation of Docker.

In the context of this project, LXC provides essential native capabilities: seamless package management for installing applications, an isolated complete filesystem, simple network isolation and, crucially, unprivileged operation. It strikes a good balance between full configuration freedom inside the environment and isolation and security from the host.

The config-file abstraction is useful here, making low-level configuration more manageable while still keeping it accessible. The configuration files are also conveniently split into a static template file, a generated per-container file, and a third one that supplies the host-specific details dynamically only at start time, perfectly decoupling the host and allowing heterogeneous use. The locations of the first two are flexible, adapting well to the portable architecture.

There are still challenges, but each comes with a highly compatible solution:

- cross-host UID/GID translation and GUI/hardware integration — LXC natively supports ID remapping via user namespaces and device and socket passthrough via config-based bind mounts, so these are solvable rather than fundamentally obstructed;
- host security modules that limit flexibility — the distribution specific ones (AppArmor on Ubuntu, SELinux on Fedora) can be disabled, while a custom, cross-compatible security measure can be built in their place.

LXD / Incus

LXD (Linux Container Daemon) is a Canonical backed, but still community based, effort to make LXC easier to configure and use, adding a hypervisor-like management daemon and a polished interface on top of LXC. However, in 2023 Canonical withdrew LXD from the community run Linux Containers project and took development fully in-house, requiring contributors to sign a Contributor License Agreement (CLA) [7]. In response, the original maintainers forked the last community release as Incus, which was adopted by the Linux Containers project and has since been packaged across major distributions [8].

Incus provides the same package management, private filesystem and flexible storage advantages as LXC, while offering an easier to use interface. It comes, however, with a core mismatch for the portable use case: its design is host-centric, keeping a centralized container database on the host (under `/var/lib/incus`). Incus does provide default GUI passthrough configurations, but these are also tightly coupled to that centralized host database. While it may be possible to work around this, doing so is difficult and unreliable, requiring hacking against Incus's fundamental host daemon and database model.

3.4. Best Suited Technology

LXC proves to offer the best balance: low-level control, essential for UID mapping, device passthrough and a host-facing read-only filesystem, while its configuration files keep that control more manageable than the bare namespace mechanisms. It also provides several mechanisms essential to this project by default, such as ID, socket and device mapping, unprivileged operation and isolation from the host. Out of most of the alternative technologies, it has the unique advantage of allowing the current use case to be built on top of its provided mechanisms, rather than having more mechanisms at hand but requiring the project to fight against deeply host dependent design principles.

3.5. Closest Current Alternative

To the knowledge of the author, there is currently no prominent, widely available direct competitor that solves the same problems the PPE attempts to, current approaches being mostly confined to the limited live USB and cloud VM options already discussed.

The closest container-based, well documented practice is using Distrobox or raw rootless Podman as a portable personal environment. Revisiting Distrobox, now not as a candidate technology, but as the closest existing practice, the project itself documents tight integration with the host, sharing the user's home directory, external storage, graphical apps and audio [9]. This approach implies creating a container whose home lives on

portable storage and relocating it between machines, typically via container images or dotfile tooling [10]. While conceptually sharing the same ambition as the PPE, it differs in key areas that define this use case:

- *Isolation vs. integration* — the PPE is built to contain the environment, with a read-only filesystem from the host's view, full drive encryption and confinement on an untrusted machine, whereas the Distrobox pattern is built for the opposite, deliberately giving the container full access to the host's home and devices and stating that sandboxing is a non-goal [9];
- *Form factor* — the PPE is a self-contained drive, usable immediately once plugged in, while the Distrobox pattern has no single ready-to-run medium, relocating the environment through images or dotfile tooling;
- *Scope* — the PPE provides a full GUI environment with GPU passthrough, whereas the documented portable practice is oriented toward terminal and development tooling;
- *Container class* — the PPE is a system container, while Distrobox is an application container wrapper. Although this may sound like only a theoretical difference, it represents a strong, dedicated foundation to build on, rather than a tool that must constantly be hacked to fight against its own purpose, making it a fundamental distinction for stability and further extensibility.

While clearly different, it is important to also note their shared ground: both run unprivileged on the host environment.

Chapter 4. Analysis and Theoretical Foundation

Having established, in the previous chapter, that LXC system containers offer the best-suited foundation for the PPE, this chapter lays out the theoretical foundation and the conceptual solution built on top of it. It first describes the general architecture of the system, how the container relates to the host and what guarantees its near-native performance, and the way the solution is laid out across the external drive and the host computer. It then decomposes the solution into its two main components, the host configuration tool and the session launcher, explaining the logic and the flow that each follows. Finally, it discusses how the whole system fits into the conventions of the Linux filesystem and analyzes the security concerns raised by the proposed design. The concern throughout is the reasoning behind the solution and the algorithms that realize it, rather than their concrete implementation, which is the subject of Chapter 5.

4.1. General Architecture

This first section builds up the architecture in three passes: it starts from the conceptual view of how a single environment relates to its host, then zooms out to how the solution is laid out on the external drive, and finally to how it inserts itself into the host computer. Each pass adds the context that the detailed component descriptions in the later sections rely on.

4.1.1. Conceptual Architecture

At its core, the PPE is an LXC container that runs inside the unprivileged user space of the host. The container has its own privileged space, in which the user acts as root, but this in-container privileged identity is remapped onto an unprivileged host identity (the UID translation mechanism) before any action reaches the host kernel. As a result, the kernel never grants those processes any real host privileges, no matter how privileged they appear from inside the environment.

Because it is a system container rather than a virtual machine, the environment shares the host's own kernel instead of booting a guest one, and needs no hypervisor between itself and the hardware. The applications running inside it issue native system calls that the host kernel serves directly, while that same kernel accesses the CPU and RAM as it would for any other process. This is what guarantees the near-native performance targeted by the objectives: a shared kernel, no hypervisor and native syscalls, without the translation overhead a fully virtualized machine would imply.

This choice of a system container, rather than an application one, is also what lets the environment behave like a complete operating system: it runs its own init, services and package manager, so that the student works inside a full Linux user space rather than a single sandboxed application.

On top of this, the graphical and hardware capabilities the environment needs are wired in through custom passthrough. The container is connected to the host's display server and to the GPU, so that graphical applications render with hardware acceleration on the host's own devices. Figure 4.1 illustrates this arrangement.

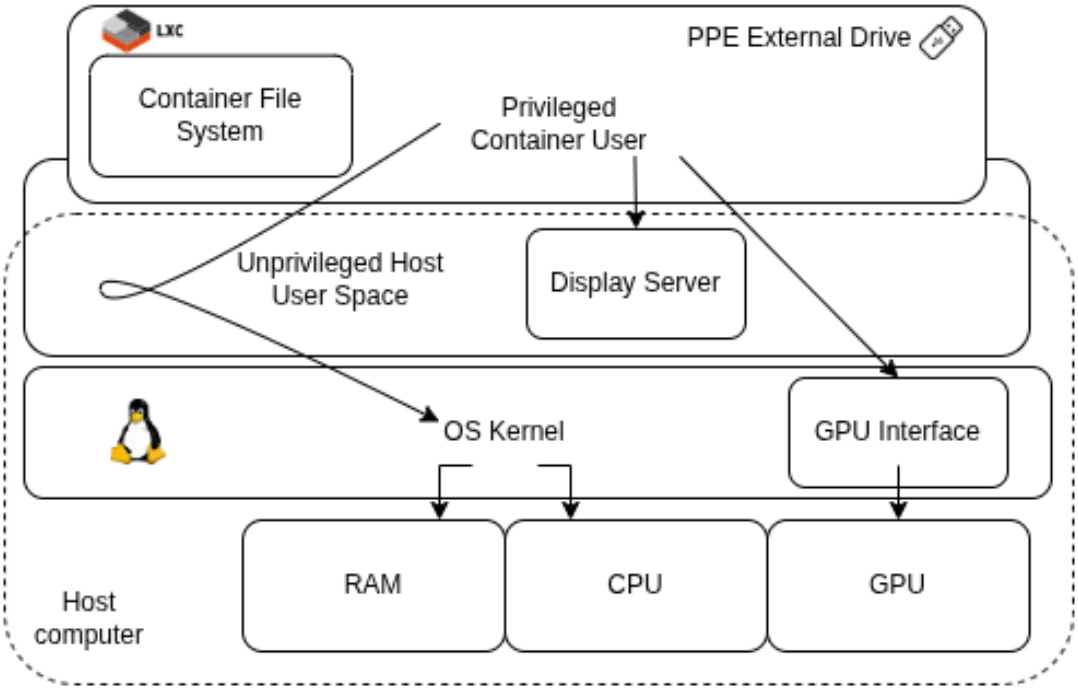


Figure 4.1: Conceptual architecture of the PPE: the LXC container running in the host's unprivileged user space, its privileged identity remapped to an unprivileged host identity before reaching the shared kernel, together with the passthrough to the host's display server and GPU.

4.1.2. External Drive Structure

The entire solution ships on the external drive and consists of two components that play complementary roles.

The *host configuration tool* runs privileged. Its job is to prepare a host computer for the PPE, configuring it so that the external drive can be mounted, accessed and used to run the containerized session.

The *session launcher* runs unprivileged. It verifies the integrity of the host configuration, then creates and configures the container, and finally launches the applications and services inside it.

The division between the two follows the privilege boundary: everything that requires administrative rights is confined to the one-time host configuration, while the recurring, everyday work of running a session is handled entirely without them.

Of everything stored on the drive, only these two components need to be present beforehand, having been loaded onto the drive when it was formatted for PPE use. Everything else, including the container itself and its configuration, is produced automatically by the processes described in the following sections. Figure 4.2 shows the blueprint of the external drive and of the container, with arrows indicating what each component creates.

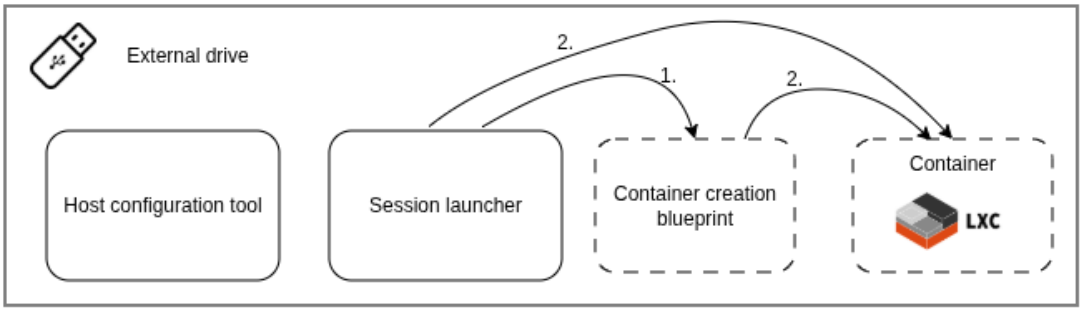


Figure 4.2: The external drive blueprint and the container it produces, indicating which component creates what.

4.1.3. Host Computer Structure

On the host side, the central mechanism is the one that makes the drive reachable and hands control over to the launcher. Rather than exposing the launcher directly, the configuration installs a desktop entry that the user interacts with, which in turn invokes a small launch stub script. That stub first checks for the existence of and mounts the external drive and only then runs the session launcher from it. This indirection is what makes the drive available on demand and hands execution over to the launcher only once the drive is in place. Figure 4.3 traces this flow, from the host's desktop entry to the launch stub and, through it, to the session launcher.

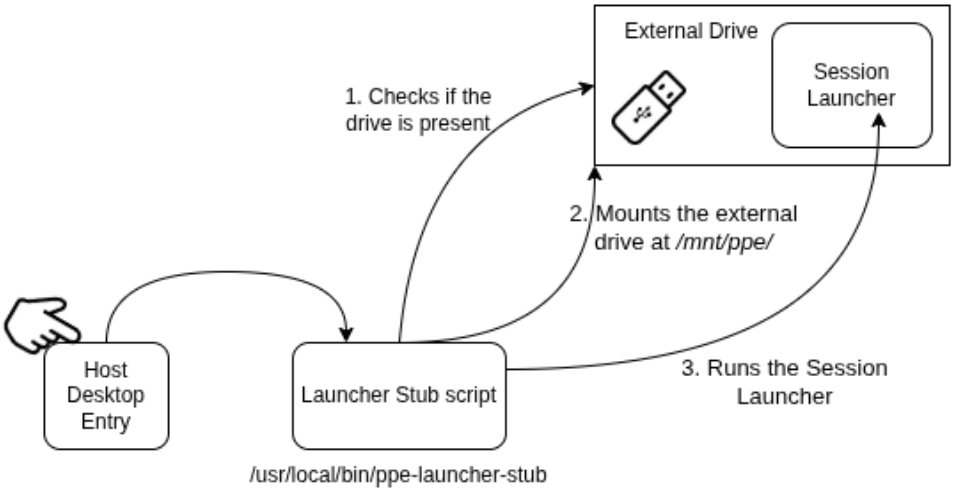


Figure 4.3: The mounting mechanism on the host: the desktop entry links to the launch stub script, which first checks if the external drive is inserted, mounts it and then runs the session launcher.

From the user's side, this whole mechanism collapses into a single familiar action: activating one desktop entry, exactly as they would launch any other application. The mounting of the drive and the handoff to the launcher happen behind that one interaction, which keeps the entry point simple and consistent across hosts.

4.2. The Host Configuration Tool

The first main component, the host configuration tool, runs with administrative privileges and prepares a host for PPE use. Before it performs any of that configuration, however, it runs a set of initial checks that decide whether it may proceed at all and against which target user it should operate.

The tool supports two complementary modes: a configuration mode, which is the default and the focus of most of this section, and a removal mode that reverses its changes. Both run privileged and share the same guarantees of safety when re-run, discussed at the end of the section. It also aims to be delivered as a single, statically-linked binary that carries no external library dependencies, so it runs on any supported host exactly as shipped, matching the portability of the drive it travels on.

4.2.1. Initial Checks

Three checks gate the rest of the process, and each must pass before any change is made to the host:

- *Privilege verification* — the tool first confirms that it is running with root privileges. Since every configuration step it performs acts on system-wide state, administrative rights are required. If they are missing, the tool warns the operator and halts without touching the host. This also reinforces the clean separation between the privileged setup and the unprivileged normal use that the rest of the system relies on.
- *Distribution family detection* — the tool reads the host's distribution identification and maps it to a distribution family (Debian, Red Hat, Arch, openSUSE or an unknown family), halting if the detected family is not supported. Currently only the Debian family, including Ubuntu, is supported, but this check aligns with the host agnostic ambition of the project: the logic is organized around a modular distribution family abstraction, so that support for further families can be added without restructuring the tool.
- *Target user selection* — finally, the tool inspects the host's regular, non-system user accounts and prompts the operator to select the one to configure. All per-user configuration is bound to this account. This aligns with the flow of an administrator configuring host computers for a single shared user account, as is currently the case in most university classrooms, while maintaining a clear separation from the privileged administrator account. The same approach also fits a student configuring a personal device for accessing their university work on the PPE, by simply targeting their own account.

4.2.2. Configuration Flow

Once the host distribution and target user have been cached from the initial checks, the tool carries out the configuration as a fixed, ordered sequence of steps:

1. *Install prerequisites* — it ensures the presence of the container runtime, together with the identity mapping and access-control-list (ACL) tooling it relies on.
2. *Permit unprivileged container networking* — it grants the selected user the right to attach the container to the host's container bridge through virtual ethernet interfaces, so that networking works without administrative rights.

3. *Prepare the mount point* — it creates the directory where the drive will be mounted, transfers its ownership to the selected user and grants the traversal rights the container needs.
4. *Grant GPU access* — it installs a persistent device-access rule that gives the container's mapped identities read/write access to the host's rendering devices.
5. *Establish on-demand mounting* — it registers a lazy mount unit linked to the drive's label and suppresses the desktop's default automounter, so that the latter cannot interfere by claiming the drive at a volatile location.
6. *Grant unprivileged mount control* — it installs a narrowly scoped, passwordless permission that lets the selected user start and stop the mount units previously declared.
7. *Install the launch stub* — it creates and places the small intermediary that checks whether the drive is present, mounts it and then hands off execution to the launcher.
8. *Register the desktop entry* — it creates the system-wide, clickable launcher entry in the standard location recognized by all desktop environments, following the freedesktop.org desktop entry convention [11].

Taken together, these steps span two kinds of change: system-wide adjustments that make the host capable of running the containerized session at all, such as installing the prerequisites and the device-access rule, and adjustments bound to the selected user, such as the networking allowance, the mount point ownership and the passwordless mount control. Once the sequence completes, the host is left ready to mount and run any PPE drive through the unprivileged launcher, with no further administrative action required for everyday use.

4.2.3. Removal Flow

The tool also supports a removal mode, whose main purpose is to clean the system so that a future version of the configuration can be applied without conflicts. Run in this mode, the tool:

1. stops the active drive mounts;
2. removes every generated artifact — the device-access rule, the automount suppression rule, the mount units, the mount permission, the launch stub and the desktop entry;
3. removes only the user-specific networking allowance, preserving any unrelated entries;
4. reloads the affected system daemons if anything changed.

Following the common practice of uninstallation processes on Linux, the packages installed during configuration are deliberately left untouched, since there is no clean way of knowing whether other mechanisms on the computer have come to rely on them.

4.2.4. Idempotency and Robustness

Both flows are built to be safe to re-run. Every step first inspects the current state and, if the corresponding configuration is already in place, only reports it and cleanly skips ahead to the next one. On the contrary, any step that fails aborts the whole chain rather than continuing past it. Together, these two properties ensure that the order in which the configurations are applied on a host can neither produce nor propagate an inconsistent state, which is what assures the robustness of this component.

4.3. The Session Launcher

The session launcher is the unprivileged component the student interacts with directly. Whereas the configuration tool prepares a host once, the launcher runs each time the student starts a session: it validates the host, brings up the personal environment and presents the interface for running applications inside it.

4.3.1. Startup Phases

Each launch proceeds through an ordered sequence of phases, each of which must succeed before the next is attempted:

1. *Host integrity check* — the launcher first verifies that the host has been properly configured for the PPE use; if it has not, it stops and reports that the host must be configured first.
2. *Environment existence and integrity check* — it then checks whether a container already exists on the drive and whether it is intact; if the environment is missing or corrupt, the launcher offers the user the option to create a new one.
3. *Container start* — with a valid environment in place, it starts the container.
4. *Start confirmation* — rather than assuming the start succeeded, it keeps polling until the container is actually functional.
5. *Personal user check* — once the container is running, it checks whether the container has been configured for personal use, that is, with an unprivileged user distinct from the default root account. If none exists yet, it guides the user through creating one and collects the corresponding credentials.
6. *Launcher presentation* — finally, with every check passed, it opens the main launcher interface.

Figure 4.4 traces this flow.

Most of these phases run automatically, surfacing to the user only as a short succession of loading states. The launcher pauses for input at just two of them: when a valid environment has to be created, and when a personal user has to be set up. Both of these are examined next.

4.3.2. Environment Creation

When no valid environment is present, the launcher provisions one in two stages.

First, it lays down the container's declarative configuration blueprint, which is responsible for the three host-facing concerns established earlier: the UID translation that remaps the container's privileged identity onto an unprivileged host one, the network attachment that connects the container to the host's bridge, and the GPU passthrough that exposes the host's rendering devices to the environment.

Second, on top of this blueprint it runs an interactive distribution selection tool, whose input determines which distribution the environment is based on and, from that choice, creates the actual container.

Because the distribution is chosen interactively at creation time rather than fixed in advance, different users can base their environments on different distributions from the very same launcher.

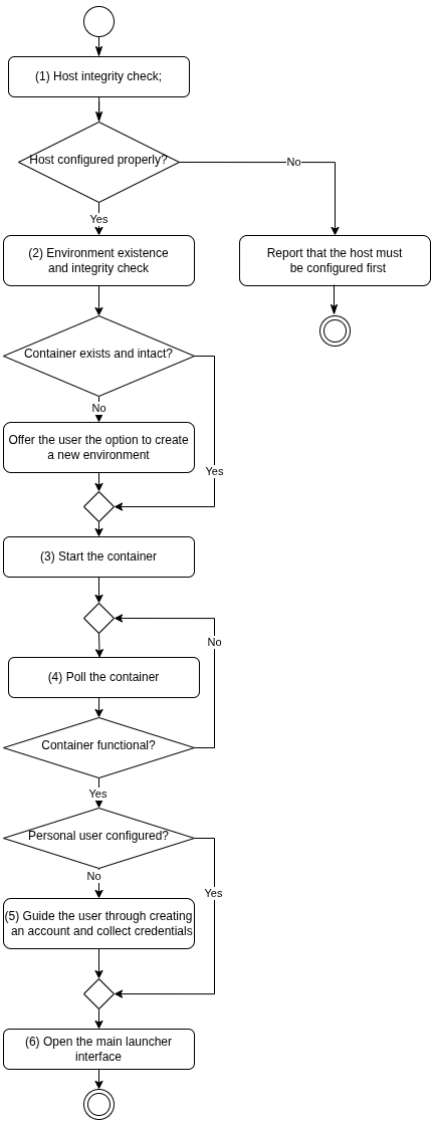


Figure 4.4: The session launcher's startup flow, from the host integrity check through environment validation, container start and personal user setup to the main launcher.

4.3.3. Host-Container Session Bridging

For the containerized environment to actually make use of the host's graphics, the launcher has to bridge the two across the host–container boundary. This bridging is not a single step but is established at three points over the course of the startup: the necessary access permissions are granted during the host verification, the display server socket is bound when the container starts, and the session environment and message bus are set up only once the personal user inside the container has been checked or configured.

The combined effect is that, on every launch, the launcher wires the host's display server socket and GPU interface through to the container and maintains a session environment that every application launched inside it inherits. The display socket, in particular, is passed in dynamically at start time rather than being written permanently into the container's configuration. This is what keeps the environment portable across hosts, since the host-specific socket is never baked into the blueprint that lives on the drive.

4.3.4. Personal User Configuration Inside the Container

By default, a freshly created container has only its root account, which is convenient for server use, but goes against the principle of least privilege: the guideline that a user should have access only to what the current task requires, so as to reduce the potential attack surface. This is precisely why almost all desktop systems are used through a non-root account that holds administrative (sudo) privileges the user can invoke only when needed and give up again immediately after.

To bring this practice into the PPE seamlessly, on every boot of the container the launcher checks that a separate, unprivileged user distinct from root is configured. On the very first boot none exists yet, so a small utility lets the user set up a new account and password directly through the graphical interface. To make sure this is a truly separate identity, the setup rejects reserved names (root, common administrative accounts and the distribution's own default users) so that the account the student creates cannot collide with a privileged or system one.

4.3.5. Main User Interface Structure

The launcher's interface is organized as a small state machine of views. Two of these are the main views the user moves between: a welcome screen, from which the container can be started or the drive ejected, and the main launcher, the grid of applications presented once the environment is running. Around them sit a number of auxiliary views covering the transitional and exceptional states: loading screens shown while operations are in progress, warning screens for the various failure conditions and the user configuration screen used for the first-run account setup described above.

4.3.6. Launcher Structure

The launcher screen itself is built from two kinds of entries: the applications automatically discovered inside the container and a set of custom built-in entries layered on top.

The discovered applications come from reading the container's freedesktop `.desktop` entries, whose locations are detailed in Section 4.4. This list is kept continuously synchronized with the container, entries appearing and disappearing on their own as applications are installed or removed, with no manual refresh required. On top of this dynamic list sit the custom built-in entries, which expose the PPE's own utilities:

- *File sharing system* — opens the dedicated shared directory, configured for effortless two-way file transfer between the host and the container.
- *Integrated terminal* — attaches a terminal to the container, giving an interactive shell for configuration work and for running command-line applications.
- *Integrated file system* — opens the host's file manager on the container's home directory, for browsing the environment's files.

Alongside these entries, the launcher provides a runtime diagnostics utility that performs on-demand integrity checks of the live environment, verifying network connectivity, GUI integration, GPU passthrough and the health of the container's own system. Finally, a stop button lets the user shut the container down, which leads into the termination flow described next.

4.3.7. Session Termination

A session can be brought to an end in two ways, both of which converge on the same safe mechanism.

The explicit path starts from the stop button on the launcher page: the launcher tries to stop the container gracefully and, if it fails to shut down in time, prompts the user to force the shutdown. Once the container is down, an eject button becomes available. Activating it unmounts the drive, after which the user waits for the notification confirming that the drive was unmounted successfully and can be safely removed.

Alternatively, the user can simply close the application window. This triggers the same forced shutdown and eject automatically, followed by the same wait for the successful unmount confirmation. Figure 4.5 traces both paths.

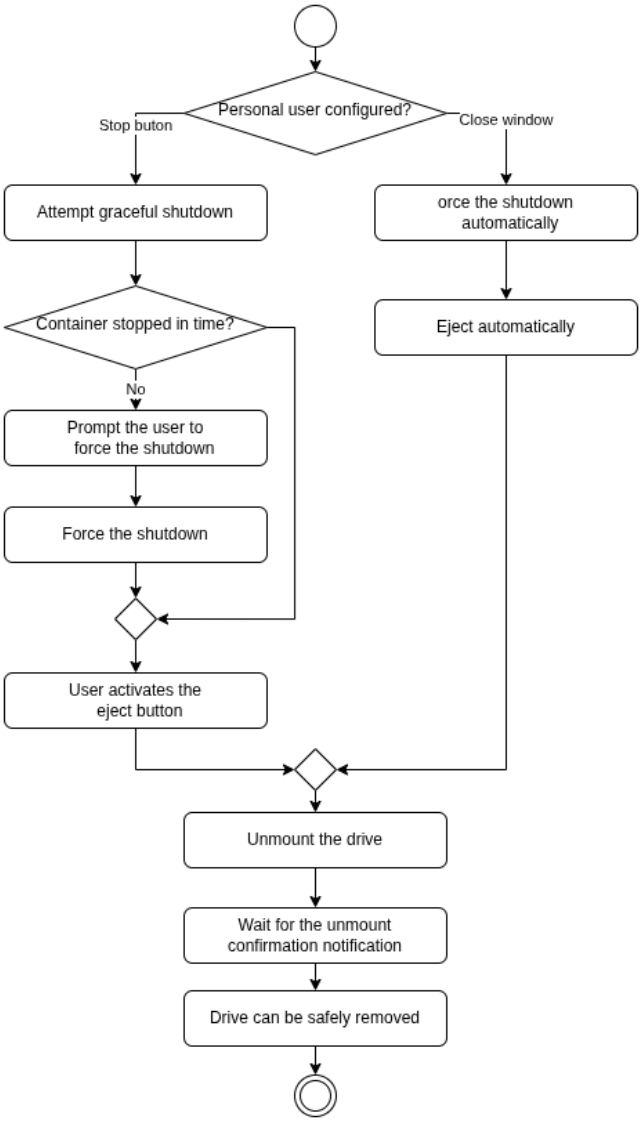


Figure 4.5: The session termination flow, showing the explicit stop-and-eject path and the window-close path converging on the same shutdown, unmount and confirmation.

4.3.8. The Unmount Mechanism

Unmounting calls for special treatment because of a self-referential problem: the launcher runs from the very drive it needs to unmount, so it cannot carry out that unmount while it is still executing from it.

The solution is to delegate the unmount to a small, separate component. The launcher writes out a short script and starts it. The script immediately enters a brief wait of two seconds, during which the launcher application itself closes and releases the drive. The script then wakes up, unmounts the drive and sends a notification informing the user that the drive can be safely removed. Figure 4.6 shows this sequence, highlighting the creation of the helper, the closing of the application and the delayed unmount that follows.

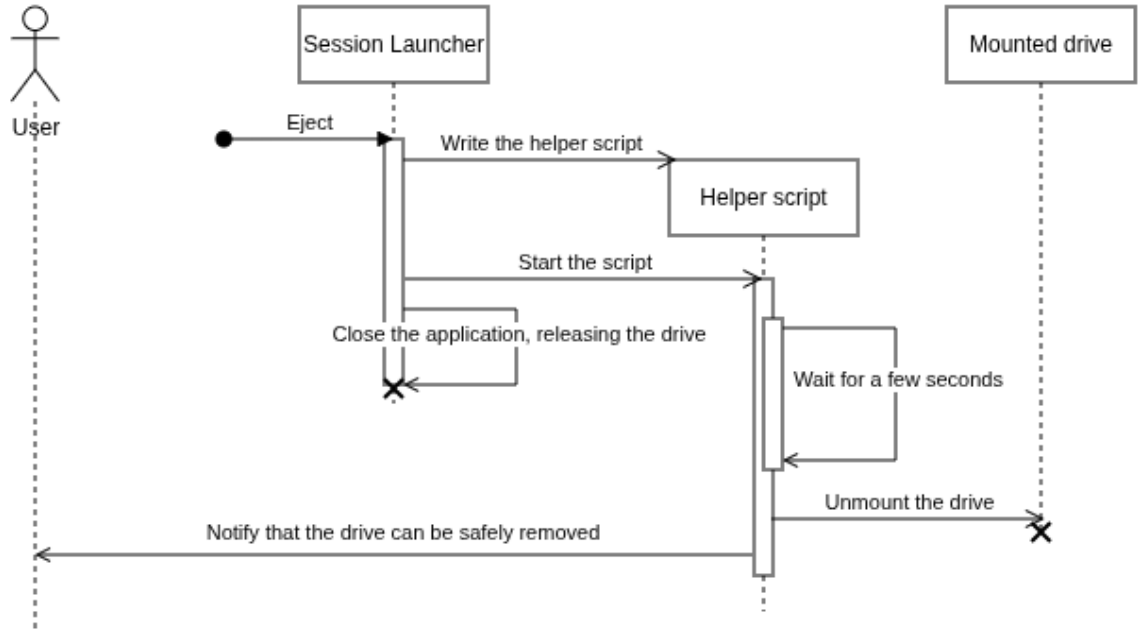


Figure 4.6: Sequence of the unmount mechanism: the launcher spawns the helper script, closes and releases the drive, after which the helper performs the unmount and notifies the user.

4.4. Integration in the Linux Filesystem

Throughout the design, both components deliberately follow the established conventions of the Linux filesystem, both to stay compatible across heterogeneous hosts and to avoid interfering with the rest of the system. This section walks through the locations that matter to the project, grouped by whether they are relevant on the host side, inside the container, or on both. Figure 4.7 highlights and numbers these locations, distinguishing the host-side ones from the container-side ones.

Two conventions apply on both sides of the boundary:

1. *Desktop entries* — the freedesktop.org desktop entry convention [11] serves a double role. On the host, the standard applications directory (`/usr/share/applications`) is where the configuration tool registers the clickable launcher entry; inside the container, the launcher reads the same `.desktop` entries, from both the system-wide and the per-user applications directories, to discover the installed programs.

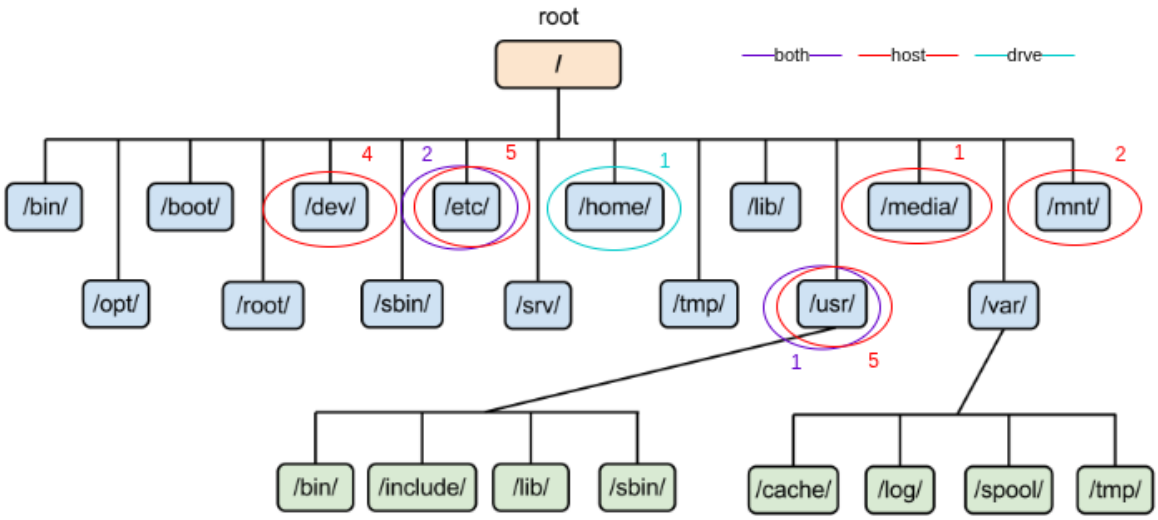


Figure 4.7: The filesystem locations relevant to the project, numbered and colored for separation between host-side and container-side.

2. *Distribution-identification file* — both components determine which distribution they are dealing with by reading the standard distribution identification file (`/etc/os-release`): the configuration tool to resolve the host's family, and the launcher to detect the container's distribution.

On the host side, the relevant locations are:

1. *Default mount point* — before the custom configuration takes over, the desktop's automounter would place the drive at its default removable media location, `/media/<username>/<drive_label>`. Because this path embeds the specific username, it varies from one host to the next, which is precisely what makes relying on the default mounting brittle across heterogeneous machines.
2. *Custom mount point* — to avoid that per-host variation, the configuration instead directs the drive to a fixed location, `/mnt/ppe`, under `/mnt`, the directory the Filesystem Hierarchy Standard reserves for temporarily mounted filesystems, so that it neither depends on the username nor conflicts with anything else on the system.
3. *Wayland display server socket* — the display server socket the environment connects to lives in the per-user runtime directory prescribed by the XDG base-directory convention (`/run/user/<uid>`).
4. *GPU interface* — the host's rendering devices, through which hardware acceleration is provided, reside under `/dev/dri`.
5. *Configuration footprint* — the persistent changes the tool leaves behind are kept in their conventional locations: the udev rules for on-demand mounting and GPU access (under `/etc/udev/rules.d`), the unprivileged networking allowance (in `/etc/lxc/lxc-usernet`) and the launch stub (under `/usr/local/bin`).

On the container side, three locations matter:

1. *Shared transfer directory* — the dedicated folder used for exchanging files with the host lives in the container user's home directory (`~/Shared`).
2. *Message-bus socket* — the session's inter-process message-bus socket is placed within the user's runtime directory, keeping inter-application communication scoped to the

user's session.
3. *Icon theme directories* — to display the applications' icons, the launcher resolves them from the container's standard icon theme directories.

By keeping to these standard locations rather than inventing its own, the project stays predictable to the host system and unobtrusive to the rest of it, and it remains portable across the distributions it aims to support: each component looks for what it needs where the conventions say it should be, regardless of the particular host or container distribution.

4.5. Security Analysis

Because the PPE is meant to run on exactly the kind of shared, multi-user and untrusted machines described in the opening chapters, classroom computers on which one user's files may be exposed to all the others, its design has to be examined against the attacks it might face there. This section carries out that evaluation, the security assessment step anticipated among the project's objectives. It works through the possible attacks on the system in turn, starting with those for which the design already provides a concrete solution and ending with the ones that remain only partially addressed or theoretical.

4.5.1. Tampering with the Mounting Rules

The design deliberately lets unprivileged users start and stop the drive's mounts, so that day-to-day use never requires administrative rights. On a shared machine, however, that same ability can be turned against others: a malicious user could stop the automounting for everyone else, making it impossible for anyone to use their own PPE drive on that computer.

The countermeasure is built into the normal launch path. Before handing off to the launcher, the launch stub always makes sure, on every launch, that the automount and mount units are active and in a functional state, clearing any leftover errors and restarting them if needed. As a result, even if another user has disabled or broken the mounts, the next launch restores them to a working state before the drive is used.

4.5.2. A Stolen Drive

Persistence is one of the PPE's defining properties: all of the user's data, applications and configuration live on the drive so that they survive across sessions and hosts. That same persistence, though, means the drive concentrates a great deal of sensitive information in a single portable object. If it is stolen, or merely left unattended and accessed by a malicious party for even a short period, all of that information could be read and leaked.

The fix is to encrypt the whole drive. This is done manually when the drive is first formatted, automatic formatting being outside the project's scope, but the complete drive encryption it enables is expected as part of preparing a drive. Once the drive is encrypted, a password is required to decrypt it before any of its data can be read, so a stolen or briefly accessed drive reveals nothing on its own. This protection naturally covers only the drive at rest: once the password has been supplied and the drive decrypted on a host, its contents become readable there, a situation examined further below.

4.5.3. In-Container Compromise Escalating to the Host

One of the environment's deliberate features is that the user holds genuine root capabilities inside their own container, which is what answers the lack of privileges on the institutional computers. This, however, raises an obvious concern: if that in-container root were also root on the host, a compromise of the environment would immediately become a compromise of the whole machine. The in-container root therefore has to be clearly separated from any real authority on the host.

This separation is exactly what the UID translation introduced earlier provides. The in-container root, together with every command it runs, is mapped onto an unprivileged host identity before it reaches the host kernel, so the processes always execute under that unprivileged identity and the kernel grants them no host privileges at all. Consequently, even if an attacker were to gain full root control inside the container, that control would still amount to nothing more than an ordinary unprivileged user on the host, keeping the compromise confined to the environment and unable to reach the host.

4.5.4. A Compromised Host Tampering with the Container

The previous cases concern a hostile user or a lost drive, while this one concerns a host that is itself compromised. It helps to separate two different goals such an attacker might have: reading the environment's data, and altering it. Once the user has supplied the decryption key on the host, there is little that can be done to stop a compromised host from reading the now decrypted sensitive files. Protecting the environment's integrity, on the other hand, preventing the host from tampering with or corrupting the drive, can be achieved entirely.

The defence relies on the same identity-mapping mechanism, and it is what realizes the environment's read-only from the host property. On a Linux filesystem, every file grants its permissions separately to its owner, to its group and to everyone else. The container owns its files under a mapped host identity that differs from the real host user and shares no group with them, so the host user falls into that last, "other" class for the container's files, which is configured without write access. As a result, no matter what a compromised host attempts, it cannot modify the environment. The single, deliberate exception is the Shared directory: it exists precisely to make file exchange between the two sides easy, and is therefore configured to be writable by both the host user and the container user.

4.5.5. A Malicious Launcher

The launcher is the component the student interacts with most directly, and over the course of a session it handles sensitive input, including the account credentials set during the environment's first run configuration. If it were replaced with a malicious, modified copy, that compromised launcher could continuously exfiltrate the user's input and activity, including that sensitive input and the user's files.

The mitigation for this is, for now, only theoretical. The idea is to digitally sign the launcher and have the launch stub verify that signature before running it, relying on the stub's existing role as the gatekeeper that hands off to the launcher. In this scheme, a private key kept on the development machine is used to produce both a signature for the launcher and a corresponding public key. The public key is embedded into the launch stub, and at launch the stub checks the launcher executable against the signature using

that public key, allowing execution to proceed only if the verification passes. A launcher that had been tampered with would fail this check and never run.

4.5.6. Human Error and Unattended Sessions

Finally comes the human factor, which lies largely outside what the software can control. A student might get distracted and leave the classroom or the area of the computer while the drive is still mounted and decrypted and the container still logged in. In that state, the safeguards discussed above have already been opened: the drive is decrypted, so the encryption no longer helps, and the session is active, so anyone nearby has direct access to the running environment and its files.

No solution can fully account for human error, and the most effective response here is thorough training against such behaviour. As a helpful software safety net on top of that, an automatic time-out that unmounts the drive after a period of inactivity could limit the attack surface, leaving the drive exposed for less time in these situations. Implementing such a mechanism comes with its own difficulties, however: reliably monitoring user activity inside the container and across its heterogeneous applications is not only hard to achieve, but would itself introduce a new point of vulnerability.

Taken as a whole, the threats that fall within the software's reach, the tampering with the mounting rules, the theft of the drive and the two directions of host–container compromise, are each met by a concrete mechanism already built into the design. What remains are one threat that is understood but for now only addressed in theory, the substitution of the launcher, and one that is ultimately human rather than technical, the unattended session. Recognizing these residual risks is an essential part for the analysis of the real feasibility of the PPE being integrated in educational institutions.

Chapter 5. Detailed Design and Implementation

While Chapter 4 laid out the conceptual foundation of the PPE and the reasoning behind its main design decisions, this chapter turns to how that design is concretely realized. It documents the developed system at the level of its actual software components, so that the two artifacts shipped on every drive, the host configuration tool and the session launcher, can be understood, maintained and extended based on these details.

The chapter opens with a design-level view of the running system and its components, then formulates the use cases that frame what the system is expected to do and for whom. From there it follows the drive itself: how it is prepared through formatting, and, in the sections that follow, how each of its two components is built and how their components fit together into a single working environment.

5.1. Design-Level Architecture Overview

The conceptual architecture in Chapter 4 described the PPE in terms of a container remapped onto an unprivileged host identity, sharing the host's kernel. This section takes the same system and looks at it as concrete software components: the pieces that actually run, and the boundaries across which they talk. Figure 5.1 expresses the overall picture.

At the level of concrete software, the session launcher, the second of the drive's two components, is built as a Tauri 2 application, a framework that pairs a native, compiled backend with a web-based frontend rendered in the host's system WebView. The two halves have clearly separated responsibilities. The backend is written in Rust and carries out all of the system-facing work, from validating the host and driving the container's lifecycle to discovering and launching applications inside it. The frontend is a React application that owns the entire user interface. The two communicate only across the Tauri boundary: the frontend issues requests to the backend, and the backend pushes asynchronous notifications back, so that the interface and the system logic stay cleanly decoupled.

The launcher does not itself contain the environment, but only manages the LXC system container that lives on the drive. Everything the student actually runs, their applications, terminals and file browsers, happens inside that container, which the launcher creates, boots and eventually shuts down.

Inside the container runs a small set of services the environment depends on, most notably among them a D-Bus session bus. This is the standard inter-process communication channel on Linux, and it also represents the storage through which applications persist their preferences across sessions, so configuring it inside the container is what lets the environment keep its settings, one of the project's defining properties. The details of how this bus is set up and kept alive are treated later in this chapter.

For graphics and connectivity, the container reaches out to two host-side services. The first is the host's Wayland display server, whose socket is passed through so that

graphical applications render on the host's own display, with GPU acceleration on the host hardware. The second is the host's network, to which the container attaches through a virtual ethernet link into the host bridge, giving the environment native connectivity. These two host services, together with the container and its own services, are exactly the passthrough connections that the conceptual architecture referred to, now seen as concrete endpoints the launcher wires together on every session.

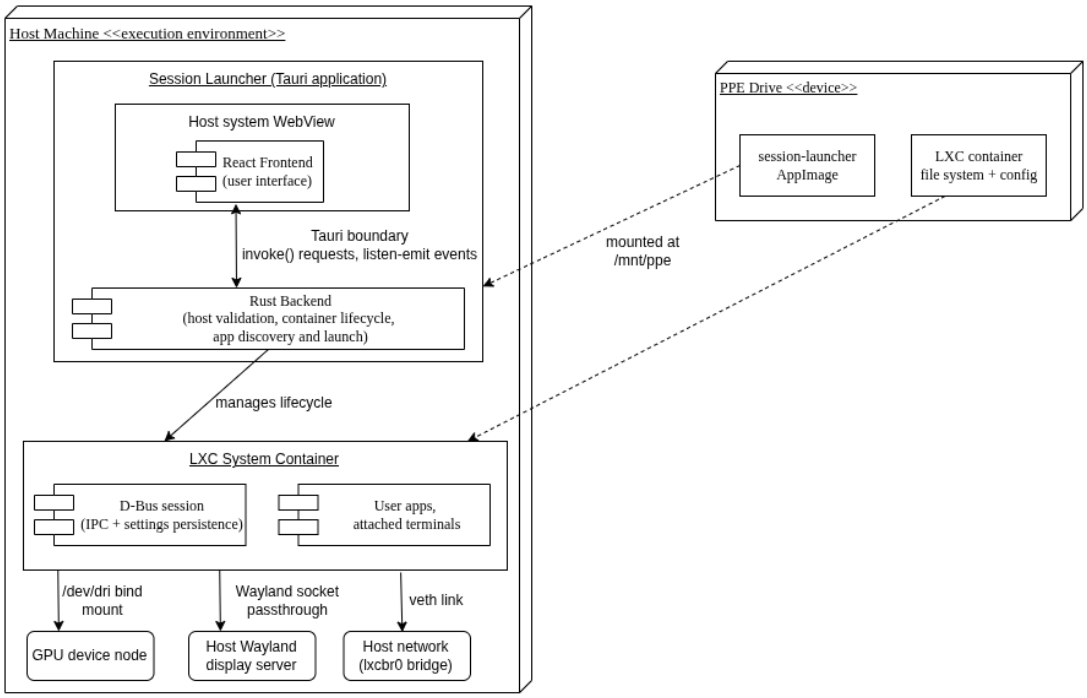


Figure 5.1: Deployment view of the running system: the Tauri session launcher (its Rust backend and React frontend rendered in the host WebView) managing the LXC container, the container and wiring it to the host's Wayland and network services.

5.2. Use Cases

Before descending into the components themselves, it is worth clarifying what the system is expected to do and who does it. This section identifies the actors, enumerates the use cases each of them drives, and then walks through the single end-to-end scenario that ties most of them together into the expected everyday use of the PPE.

5.2.1. Actors

Two actors interact with the system. The *student* is the primary actor: the person the PPE is built for, who carries the drive, runs the environment and does their work inside it. The *university classroom administrator* is a supporting actor, responsible for preparing the shared classroom computers for PPE use through the privileged host configuration tool.

The split between them follows the privilege boundary drawn in the previous chapters: everything requiring administrative rights on the host is confined to the administrator's one-time configuration, while the recurring, everyday session belongs entirely to

the student. On a personal device, the same person naturally plays both roles, running the configuration tool once against their own account and on their own machine and then using the environment as an ordinary student would.

5.2.2. Use Case Inventory

The identified use cases group cleanly by the tool through which they are performed. Those exercised through the host configuration tool, which must be run with root privileges, are shared between the classroom administrator on the institution's computers and the student on their own personal machine:

- *Configure a computer host for PPE use* — prepare a host so the drive can be mounted, accessed and run.
- *Remove the PPE configuration from a host* — reverse that configuration, cleaning up the host.
- *Consult the tool's usage* — review how the tool is used, through its `--help` flag.

All the remaining use cases belong to the student and are exercised through the session launcher. They fall into three natural stages. The first is the main session lifecycle:

- *Launch the PPE session* — plug in the drive, unlock it with the decryption password, and open the launcher through its desktop entry. The launcher verifies the host's integrity before allowing use.
- *Start the container* — boot the personal environment on the validated host.
- *Stop the container* — shut the environment down gracefully, or forcefully if the graceful stop fails.
- *Unmount and unplug the drive* — release the drive explicitly, or implicitly by closing the launcher window.

The second stage covers setting the environment up, which happens the first time a drive is used or when something is found to be compromised:

- *Create the container* — interactively choose the distribution and version to build.
- *Repair a corrupt environment* — detect an incomplete or damaged environment and recreate it.
- *Set up the container user* — on the first run, create the personal unprivileged account with a chosen username and password.

The third stage is the in-session use itself, the actions available once the environment is running:

- *Launch a GUI application* — run one of the environment's graphical applications as a standalone window.
- *Launch an application with an attached terminal* — run it with its terminal output visible, for debugging through the logs or for command-line applications.
- *Open a container terminal* — open a terminal emulator with the environment's shell attached.
- *Install applications inside the environment* — using the container distribution's native package manager.
- *Browse container files* — open the host file manager on the environment's filesystem.
- *Exchange files with the host* — open the transfer folder in the host's file manager and transfer files both ways.
- *Search for an application* — filter the launcher grid to quickly locate an application.
- *Run runtime integrity diagnostics* — check that the live environment's subsystems are healthy and working.

Figure 5.2 presents these use cases together with the system boundary and the two actors.



Figure 5.2: Use case diagram of the PPE, showing the student and the classroom administrator, with the use cases of each, against the system boundary.

5.2.3. Central Usage Scenario

Composed of several of the atomic use cases above, the central usage scenario is the main end-to-end flow that PPE users follow. In it, a student plugs their external drive into a classroom host computer in order to work on a project by accessing their own applications and files. The primary actor is the student, and the classroom administrator remains a supporting actor throughout.

The basic flow begins with the student plugging their PPE drive into the classroom host and providing the decryption password, and proceeds as follows:

1. The system performs the real-time decryption of the drive.
2. The user launches the PPE desktop entry.
3. The launcher tests the integrity of the host.

4. The user starts the container.
5. The launcher gathers and displays all the installed applications.
6. The user runs the desired applications.
7. The system saves all files modified by the user.
8. The user shuts down the container and unmounts the drive.

The flow ends with the student safely removing the external drive from the host computer.

Two alternative flows branch off this basic path. The first handles a host whose configuration is corrupt. Starting after step 3, once the launcher determines that part of the host computer's configuration is corrupted:

1. The user is presented with an error message describing the exact problems found in the host's configuration.
2. The user asks the IT administrator for assistance.
3. The IT administrator runs the host configuration utility with host root privileges.

The alternative flow then returns to basic flow step 3, letting the launcher check the host's configuration once again.

The second alternative flow handles the case where a needed application is not yet installed in the PPE. Starting after step 5, when the user cannot find the application they want to use:

1. The user looks for the desired application in the distribution's package manager.
2. The user enters their root credentials for the isolated environment to proceed with the installation.

The alternative flow then rejoins the basic flow at step 6.

The scenario carries two preconditions: the host computer is already powered on, and it has been previously configured for PPE use. On completion, it guarantees two postconditions: the host computer is ready to be used by another student, and the work done by the primary actor is saved on their portable system.

5.3. Drive Formatting

Everything described so far assumes a drive that has already been prepared for PPE use. That preparation is done manually: as stated among the project's scope boundaries, automatic formatting of a new drive is deliberately left out of scope, so a drive is made ready by formatting it and loading the two components onto it by hand. This section describes what that formatting entails, since the choices made here form the foundation for several of the properties the rest of the system relies on.

The drive is formatted with the *ext4* filesystem, and this choice is not arbitrary. Being a native Linux filesystem, ext4 preserves POSIX ownership and permissions, which matters in two distinct ways. First, it is what allows the access-rights model behind the environment's read-only from the host perspective protection to hold on the drive at all: the ownership and permission bits that keep the host user out of the container's files, discussed in Chapter 4, only survive on a filesystem that stores them. Second, it keeps the two shipped components, the configuration tool and the launcher, executable across different hosts, since their executable bit is retained rather than lost as it would be on a filesystem that does not carry Unix permissions.

The drive is then labelled with the standard name `PPE`. This label is not cosmetic, it is the identifier by which the drive is recognized throughout the host side of the system.

The launch stub, the on-demand mount unit and the automount mechanism all locate the drive by this exact label, so a drive named anything else would simply not be found by the configured host.

Finally, the drive is configured with full encryption, with a password set at format time. The mechanism used is *LUKS*, the standard drive encryption procedure on Linux, supported by default across all distributions. Encrypting the whole drive is what answers the stolen drive threat raised in the security analysis of Chapter 4: because the drive holds all of the user's persistent data and configuration, encryption ensures that a lost or briefly accessed drive reveals nothing without the password. This protection comes at almost no cost to performance, because LUKS makes use of the CPU's AES-NI hardware instruction set, present in essentially every x86-64 host since around 2010 (Intel from the Westmere microarchitecture [12], and AMD from Bulldozer in 2011 [13]), so that encryption and decryption run in real time with negligible overhead.

5.4. The Configuration Tool

The conceptual behaviour of the host configuration tool, its initial checks, the ordered sequence of configuration steps, the removal mode and the guarantees that make it safe to re-run, was laid out in Chapter 4. This section documents how that behaviour is concretely implemented: the technologies its two pieces are built on, the data model that structures the work, the orchestration that drives it, and the exact changes each step makes to the host.

5.4.1. Technology

The configuration tool itself is a Rust command-line tool, compiled against the musl C library so that it links statically into a single self-contained executable that runs on any supported host without external dependencies. Static linking here means that every library the tool depends on is baked into the binary at compile time, rather than being looked up on the host when the tool is run. Nothing therefore has to be already installed on a given machine for it to work. The musl library is chosen for exactly this purpose, being a lightweight C library well suited to fully static builds, where the more common default is difficult to link in this way. The payoff is a binary that behaves identically no matter which distribution or library versions the host happens to use, matching both the host agnostic ambition of the project and the portability of the drive it travels on.

The decision to keep it a command-line tool follows directly from that same need to statically bundle all of its libraries for cross-platform usability. Self-contained graphical libraries would take up much greater space. Since the convenience of a tool that, although essential, must be run only rarely is not a high priority, and since its use involves almost no interaction beyond running it and selecting the target user, this compromise is well justified for the benefit of freeing up precious external drive space.

The launcher stub that the tool installs on the host is, by contrast, a small Bash script. Because its use is completely automated within the execution flow, there are no usability constraints for it, so its only real objectives are ease of deployment and cross-platform compatibility. A Bash script can be executed on any Linux host by default, without requiring any further dependencies or configuration, and it needs no compilation, being interpreted at runtime and deployable as a simple file with the correct permissions. Given all this, the choice of technology for the stub becomes obvious.

5.4.2. Data Modeling

The tool's work is structured around a small vocabulary of types shared by every module: the `DistroFamily` enumeration, the `TaskStatus` enumeration, the `Task` struct and the `Result<TaskStatus, String>` type they combine into.

The `DistroFamily` type cohesively groups everything related to distribution management: the list of all distribution families, the logic for grouping a detected system into one of them, and the decision of which families are currently supported. While it is neither strictly a class nor a plain enumeration, Rust allows a mix between the two, enriching the clear set of distribution families of the enum with self-referencing methods and constructors. The four identified families are Debian, Red Hat, Arch and openSUSE, alongside an unknown entry that simply carries the detected string when none of the four match.

The `TaskStatus` enumeration has only two values, `ActionCompleted` and `JustChecked`, but it is essential for modeling the idempotent nature of the configuration actions. Each action is implemented in two phases: first checking whether the configuration has already been performed, and only actually performing it if necessary. Rather than cluttering the code with repetitive conditional logic, abstracting the execution result into this enumeration brings benefits both in the clean early termination of the configuration steps and in enforcing the idempotency that characterizes the whole configuration.

Leveraging another Rust mechanism, the `Result` type allows for clean handling of the complex return messages coming out of the execution modules. Here, having every task return a `Result<TaskStatus, String>` cleanly encapsulates both the outcome of the action performed and any additional justification, so that the message eventually displayed to the user is easily streamlined, without the need for extra conditional statements or parsing logic.

All of this comes together in the `Task` struct, which imposes a clear, uniform format on every task, regardless of whether it belongs to the configuration or the cleanup process: a description, used for both the developer and the user and so ensuring the transparency of this privileged process, together with the action, a Rust function type that fixes the arguments and return type of the step. In practice, this means that every task accepts the host `DistroFamily` and the target user it must configure as arguments, and returns the `Result<TaskStatus, String>` type described above. Listing 5.1 shows the core of this model, and Figure 5.3 presents all of these types together.

```
pub enum TaskStatus {
    ActionCompleted,
    JustChecked,
}

pub struct Task {
    pub description: &'static str,
    pub action: fn(&DistroFamily, &str) -> Result<TaskStatus, String
        >,
}
```

Listing 5.1: The core data model: the `Task` struct, pairing a description with a uniform action signature, and the `TaskStatus` enumeration the action returns.

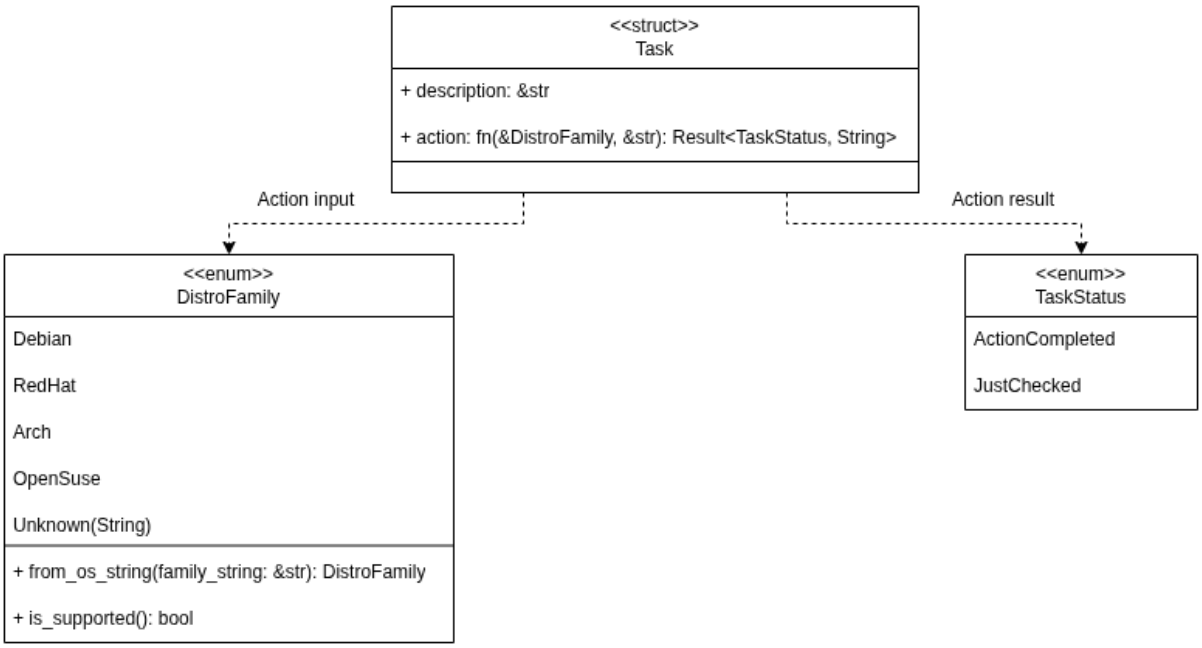


Figure 5.3: The class diagram of the configuration tool, defining the data model shared by every step.

5.4.3. The CLI and Orchestration Layer

The tool's entry point parses the command-line arguments to select between two symmetric modes: the default configuration run, or the reversal run requested through the `--remove` flag, while the standard `--help` flag simply prints the usage guide and exits. The two modes are symmetric in the sense that they are not two separate programs with separate machinery: both feed the same task-driven pipeline, and differ only in which list of tasks they hand it. The configuration mode hands it the full ordered sequence of steps, whereas the removal mode hands it a shorter list that mirrors the configuration in reverse intent, undoing changes rather than applying them. Keeping both on a single pipeline means that the progress reporting, the error handling and the safety guarantees discussed throughout this section are written once and shared by both, rather than duplicated and kept in step by hand.

Before any privileged action is dispatched, a series of gating checks guards the process, whose rationale was set out in Chapter 4. The first confirms that the tool is running as root: it reads the caller's effective user identity through a direct `geteuid` system call and requires it to be that of the superuser, since every step that follows alters system-wide state that only root is permitted to touch. The second identifies the host's distribution family by reading `/etc/os-release`, the standard file every modern Linux distribution uses to describe itself, and refuses to continue if that family is not among the supported ones. Both checks are placed before anything is written to the host, so that an unsuitable machine, one lacking the privilege or running an unsupported system, is turned away untouched rather than left half-configured.

Only once these pass does the tool resolve the target user account. It scans `/etc/passwd` for human accounts, distinguishing them by their user identifier: on Linux, identifiers below 1000 are reserved for system and service accounts and 65534 is the placeholder `nobody` account, so the 1000–65533 range is exactly the set of ordinary login users. The

operator is then prompted to choose one of them, because every per-user part of the configuration that follows, the networking allowance, the ownership of the mount point and the passwordless mount permission, is bound to that specific account and would be meaningless without knowing it in advance.

5.4.4. Implementation Details

The execution structure consists of an orchestrator that dynamically iterates through the array of tasks, injecting the host distribution and user context into each, running it and passing the result back to the user. This design carries the practical advantage of easy extensibility, both for adding new configurations and for modifying the existing ones.

Presented here in their fixed execution order, with the purpose of each detailed in Chapter 4, the concrete implementation of the eight steps is as follows:

1. *Installing prerequisites* — before any of them is installed, the tool first asks the system which of the required packages (the LXC runtime and templates, the `uidmap` identity mapping tools and `acl`) are already present, querying each one with `dpkg -s`, the command that inspects the Debian package database. Only the ones reported as missing are then installed, through a non-interactive `apt-get install`: the Debian package manager is told to proceed on its own, without stopping to ask the operator to confirm anything, so that the whole step can run unattended. This currently targets Debian alone, as that is the only supported distribution family, but the query and the installation are both routed through the distribution family helpers, so that support for another family later amounts to supplying its own package manager equivalents rather than rewriting the step.
2. *Permitting unprivileged networking* — appends one `"<user> veth lxcbr0 10"` line to `/etc/lxc/lxc-usernet`, the file through which LXC records which unprivileged users may create container network interfaces and how many. A *veth* (virtual ethernet) device is a pair of linked virtual interfaces that behave like the two ends of a cable: one end sits inside the container and the other attaches to the host bridge, `lxcbr0`, a virtual switch on the host that the container plugs into to reach the outside network. The line therefore authorizes the target user to attach up to ten such interfaces to that bridge, which is what lets the container obtain network connectivity later on without any administrative rights.
3. *Preparing the mount point* — creates the directory `/mnt/ppe` where the drive will later appear, transfers its ownership to the target user so that the user, rather than only root, may manage it, and sets `o+x` on it. On a directory, the execute bit does not mean "run" as it would on a file; it grants *traversal*, the right to pass through the directory to reach what lies inside it, as opposed to the read bit, which grants listing its contents. Adding it for "other" (`o+x`) therefore opens that traversal to identities outside the owner, including the container's mapped identity, so that the environment can reach the drive's contents through the mount point. Because that same directory is the exact path the automounter watches, merely touching it while the automounter is armed could trigger it and interfere. The step therefore momentarily stops the automount unit if it is active, verifies the ownership and permissions through the filesystem metadata, and starts it again afterwards.
4. *Granting GPU access* — writes a udev rule to `/etc/udev/rules.d/99-lxc-gpu.rules`. *udev* is the host subsystem that reacts to devices as they appear, and a rule tells it to run a given action for any device that matches. Here, the rule grants the

container's mapped identities access to the DRM (Direct Rendering Manager) render devices, the kernel's interface to the GPU exposed as device files, through an access-control-list entry that extends read/write to those identities beyond the file's ordinary owner-group-other permissions. udev is then reloaded, so that it becomes aware of the new rule, and triggered, so that the rule is re-applied to the GPU devices that are already present rather than only to ones connected in the future. This is what lets the access take effect immediately, without a reboot.

5. *Establishing on-demand mounting* — writes three files: the udev rule that suppresses the desktop's default automounting of the drive, plus the systemd mount and automount units keyed to the drive label. systemd, the host's service manager, can model a mounted filesystem as a *mount* unit and pair it with an *automount* unit that mounts it lazily. The mount unit declares how and where the drive is mounted, finding it by its `PPE` label rather than by a device path (which can change from one plug-in to the next), and giving the `/mnt/ppe` target and the filesystem type; the automount unit watches that mount point and triggers the mount only on first access, which is what provides the lazy, on-demand behaviour instead of mounting eagerly at boot. The udev rule, in turn, is needed because desktops normally claim any removable drive and auto-mount it to a volatile per-session location, which would seize the drive before these units and place it at an unpredictable path. The rule tags the `PPE` drive so that this default mechanism ignores it. Finally the step reloads udev and runs `systemctl daemon-reload`, which makes systemd re-read its unit files and recognize the new units, so that `/mnt/ppe` populates on demand whenever the PPE drive is present.
6. *Granting unprivileged mount control* — writes a narrowly scoped sudoers drop-in at `/etc/sudoers.d/99-ppe-mount`. A sudoers drop-in is a small file placed in `/etc/-sudoers.d` that extends the sudo policy without editing the main configuration, and this one grants the target user *passwordless* permission (`NOPASSWD`), meaning the user may run the listed commands through sudo without being asked for a password. Those commands are only `systemctl start`, `stop` and `reset-failed` on the PPE mount units and nothing else, so the grant is deliberately too narrow to be turned to any purpose beyond mounting and unmounting the drive. The file is then set to `440`. A Unix file's permissions are written as an octal number whose three digits set what the owner, the owner's group and everyone else may do, each digit combining read (4), write (2) and execute (1); `440` therefore grants read to the owner and group and nothing to anyone else, and, importantly, leaves the file writable by no one. sudo insists on this: it refuses to load any sudoers file that is writable by its group or by others, precisely so that an ordinary user cannot edit the rule to grant themselves broader privileges, and setting `440` is what satisfies that requirement.
7. *Installing the launch stub* — writes the launcher stub script, the intermediary the desktop entry invokes at runtime, to `/usr/local/bin/ppe-launcher-stub` and sets it to `755`. The file is owned by root, since the tool that writes it runs as root, and `755` gives that owner read, write and execute while giving the group and everyone else read and execute but not write. In practice this means every user on the machine can read and run the stub, which is exactly what the desktop entry must be able to do on each launch, while only root can modify it. On a shared computer this is what prevents a malicious user from rewriting the stub, the very script that runs on every launch, to do something harmful.

8. *Registering the desktop entry* — writes the desktop file to `/usr/share/applications/ppe-launcher.desktop` with `644` permissions and then refreshes the desktop database. Again the file is owned by root, and `644` gives the owner read and write while giving the group and everyone else read only. This is precisely what makes it tamper-proof on a shared device: every user can read the entry, which is what lets the desktop display and launch it, but none of them except root can write to it, so a regular user can use the entry but cannot alter where it points or what it runs. Refreshing the desktop database then tells the desktop environment to update its cache of known applications, so that the new entry appears among them. The result is the clickable launcher entry recognized across desktop environments.

Of these, the fixed content files, the GPU udev rule, the three on-demand-mounting files, the launch stub and the desktop entry, write their content from the corresponding constants in the templates module, while the two user-specific artifacts, the veth networking line and the sudoers drop-in, are instead generated inline, since they embed the selected username. Regardless of how its content is produced, every step reports its outcome through the idempotent checking-first pattern mentioned earlier: it first inspects what is already on the host and, if that already matches what the step would produce, reports the configuration as present and moves on without rewriting anything, so that re-running the tool never disturbs a host that is already correctly set up.

5.4.5. The Templates Module

Every fixed content file the configurator writes to the host is kept as a static string constant in a dedicated templates module, cleanly separated from the logic that places it: the GPU access udev rule, the automount-suppression udev rule, the systemd mount and automount units, the launcher stub script and the desktop entry. The two files whose content depends on the target user, the veth networking line and the passwordless mount sudoers drop-in, fall outside this module and are built inline where they are written, precisely because they embed the chosen username. Isolating the generated content from the placement logic keeps the pipeline readable and its effect on the host easy to verify.

Beyond mere organization, this separation is also what underpins the idempotency of these steps: each one compares the file already present on disk against its corresponding template constant, reporting the system as already configured (`JustChecked`) when they match, or writing the file (`ActionCompleted`) when they differ or it is missing. The template constants thus serve both as the source of the written content and as the reference against which the current host state is verified.

5.5. The Session Launcher

The session launcher is the second of the drive's two components, and the one the student interacts with directly. Its conceptual behaviour, the startup phases, the environment creation, the session bridging, the personal user configuration and the shutdown, was described in Chapter 4. This section documents how it is implemented. As introduced in the architecture overview at the start of this chapter, the launcher is a Tauri application split into a Rust backend that performs all of the system-facing work and a React frontend that owns the interface, the two communicating only across the Tauri boundary. This first section covers the backend, the frontend being treated in the section that follows.

5.5.1. Technology

The launcher is distributed as an *AppImage*. An AppImage is a single-file application format that packages a program together with all of the libraries and dependencies it needs, so that it runs on any Linux distribution without being installed and without relying on the host to provide those libraries of any particular version. This is chosen for the purpose of heterogeneous host compatibility: the launcher must run unchanged on whatever distribution a given classroom computer happens to use, and bundling everything it depends on into one self-contained file is what guarantees that. It is, moreover, the standard format for cross-distribution application distribution on Linux. In this sense the AppImage plays the same role for the graphical launcher that the static musl build played for the configuration tool: it is the form in which a graphical application achieves the same host agnostic portability as the drive it travels on.

5.5.2. Module Decomposition

The backend is organized, under `src-tauri/src`, into a set of single-purpose modules. The *checks* module holds all of the validation logic, *configure* holds the provisioning and the host-side wiring, *session* holds the runtime lifecycle, the launching of applications and their discovery, *utils* gathers the shared helpers and the detection of host tools, and *templates* keeps the static configuration strings. Tying them together is `lib.rs`, the assembly point at which the state and the full list of commands the frontend may call are registered with the framework, making it the authoritative index of the interface between the two halves of the application.

This decomposition is deliberate. Splitting the backend by responsibility mirrors the startup phases laid out in Chapter 4 onto the code itself, keeps the management of the host and the container cleanly isolated from the commands that face the user interface, and keeps the resulting command surface easy to navigate and to extend.

5.5.3. Managed State and the Singleton Pattern

Four pieces of session-scoped state are registered with the Tauri framework once, at startup, and then cached after being identified for the first time. Two of them, the *TerminalManager* and the *FileManager*, stand in for the host's terminal emulator and file manager, and exist to integrate the container's console and filesystem seamlessly into whatever utilities the host already provides. Since identifying those utilities means probing the host against a list of the common terminal emulators and file managers, and since they are needed repeatedly over the course of a session, repeating that exhaustive probing on every use would be needlessly expensive. It is avoided by finding each utility once and then caching it, exposing it globally through the *singleton* design pattern, the pattern that keeps a single shared instance of some data and hands that same instance to every caller. The lookup is moreover *lazy*: on the first request the singleton inspects its own cached field and performs the exhaustive probing only if it is still empty, so the cost is paid at most once, and only if the utility is actually needed.

The remaining two, the *SessionState* and the *AppWatcherState*, are managed as singletons in the same way, but rather than standing in for a host utility they serve as shared holders of live session state across commands: the active container username in the former, and the handle that keeps the filesystem watcher alive in the latter.

Because Tauri commands can run concurrently, and because Rust, as a safety-first language, enforces concurrency safety at compile time rather than leaving it to run-time discipline, all of this shared mutable state, the cached user, the watcher handle and the detected host tools, is placed behind *mutexes*. A mutex (mutual-exclusion lock) permits only one thread at a time to reach the value it guards, so that concurrent commands cannot corrupt it. This mechanism pairs especially well with the globally accessible, cached architecture described above, which is exactly the kind of shared state that would otherwise be unsafe to touch from several commands at once.

It is worth noting that these are not singletons in the classical sense of being declared somewhere in the code and then reached for directly wherever they are needed. Their creation and their management are instead handed to Tauri's own *managed state* mechanism, a form of dependency injection. Each is constructed once and passed to the framework to own, and any command that needs one must declare it in its signature, whereupon the framework injects a reference to the managed instance for it to use. Access is therefore explicit and mediated by the framework, rather than global in the unrestricted sense.

5.5.4. Host Tool Abstraction

Beyond finding and caching the host utilities, the *TerminalManager* and the *FileManager* also abstract the entire mechanism of calling them. Since one of the launcher's central responsibilities is precisely to act as a wrapper around the complex terminal commands that interact with the LXC container, these two components capture that responsibility directly at the design level.

The *TerminalManager* abstracts the commands, with their various arguments and flags, for launching an application from within the container with a terminal attached, whether to inspect its output for debugging or to run command-line applications. It does so both synchronously, as needed for essential configuration steps such as the initial interactive creation of the container, and asynchronously, as is the case most of the time. The one exception is the ordinary detached GUI launch, with no visible terminal, which is spawned directly and does not pass through this component. These behaviours are exposed as a few convenient public methods, while the complex logic behind them stays self-contained and easy to extend for the wide range of heterogeneous hosts, each with its own supported utilities.

The set of terminal emulators it probes for spans the defaults of the major desktop environments alongside a few popular custom ones, ending in a distribution-provided fallback:

```
let terminals = [
    "gnome-terminal",       // Standard GNOME
    "konsole",              // Standard KDE
    "xfce4-terminal",       // Standard XFCE
    "alacritty",            // Popular custom
    "kitty",                // Popular custom
    "x-terminal-emulator", // Debian/Ubuntu default wrapper
];
```

Listing 5.2: The ordered list of terminal emulators probed for on the host.

The command it then builds differs between emulators, since they do not share a single syntax for running a command inside a new window:

```
let mut cmd = match terminal.as_str() {
    "gnome-terminal" => {
        // gnome-terminal deprecated the -e flag in favor of --
        let mut c = Command::new(terminal);
        c.arg("--wait").arg("--").arg("bash").arg("-c").arg(command);
        c
    },
    "alacritty" | "kitty" | "konsole" => {
        let mut c = Command::new(terminal);
        c.arg("-e").arg("bash").arg("-c").arg(command);
        c
    },
    _ => {
        // fallback (x-terminal-emulator, xfce4-terminal)
        let mut c = Command::new(terminal);
        c.arg("-e").arg(command);
        c
    }
};
```

Listing 5.3: Building the launch command according to the detected terminal emulator.

The same construction is offered in a blocking and a non-blocking form. The blocking one finishes by waiting for the terminal window to close, which is what pauses the interface while the user works through the interactive container-creation menus, whereas the non-blocking one merely spawns the window and returns immediately.

The *FileManager* follows the very same design for the host's file explorer, abstracting the attaching of the container's file browser and its file transfer directory into the host's native file manager. Again, the supported managers are probed against a fixed list, ordered from the desktop defaults down to a universal fallback:

```
let managers = [
    "nautilus",  // GNOME
    "dolphin",   // KDE
    "thunar",    // XFCE
    "nemo",      // Cinnamon
    "pcmanfm",   // LXDE/LXQt
    "xdg-open"   // Standard Linux fallback
];
```

Listing 5.4: The ordered list of file managers probed for on the host.

The command that opens a directory in the detected manager is then kept behind a single, globally accessible method, which simply hands the chosen path to that manager and spawns it detached:

```
let mut cmd = Command::new(&manager);
cmd.arg(path);
clean_command_env(&mut cmd);
let status = cmd.spawn(); // spawn detached
```

Listing 5.5: Opening a directory in the detected host file manager.

Figure 5.4 shows these two host tool abstractions together.

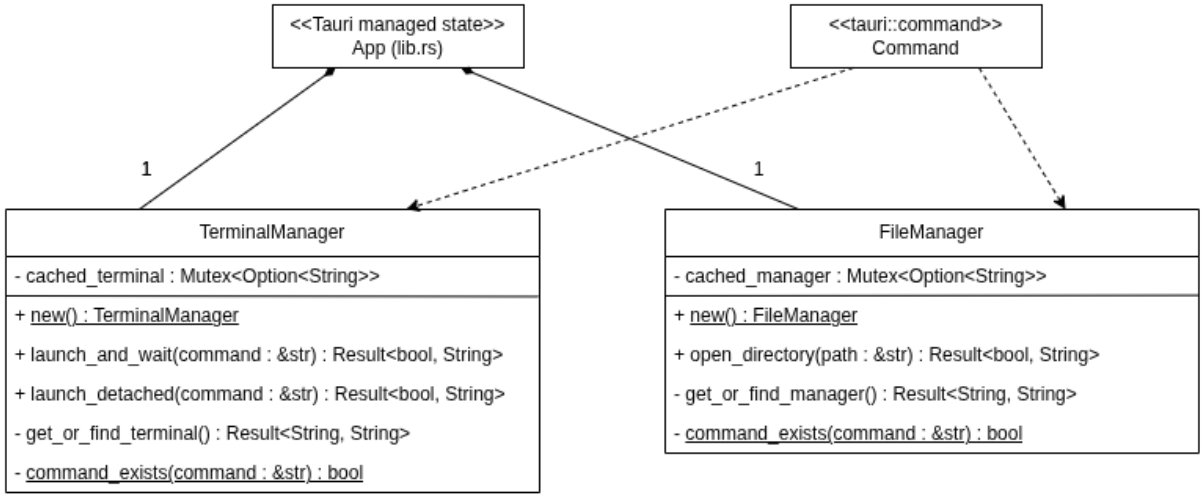


Figure 5.4: The two host tool abstractions, the *TerminalManager* and the *FileManager*, each caching the detected host utility and exposing its calling logic behind a small set of public methods.

5.5.5. The Centralized Container Execution Layer

Every action that must take place *inside* the container is funneled through a single helper, *run_lxc_shell_cmd()*. It wraps the container-entering command `lxc-attach`, running an arbitrary shell command inside the running `personal-container`, in the form `lxc-attach -n personal-container -P /mnt/ppe/lxc-data -- sh -c <cmd>`, and returning its standard output on success or its standard error on failure, uniformly, as a Rust `Result`. Every container-related call in the checks, configure and session modules passes through it, from installing packages and creating the user to the runtime health probes and the discovery of applications.

Concentrating all of this in one function is what makes it a unified point of communication between the host and the container. That confined boundary is precisely what allows the error handling across it to remain consistent, since every command that crosses into the container does so through the same success or error contract, rather than each caller inventing its own.

5.5.6. Live Application Discovery

One of the first things the launcher does once the container user checks pass is to display the grid of the student's applications, which means those applications must first be discovered. This is done by following the same freedesktop.org location convention referenced in Chapter 4: the launcher scans the container's application directories, the system-wide `/usr/share/applications` and the user's own `~/.local/share/applications`, both reached through the container's root filesystem on the drive, for `.desktop` entries. Each found entry, describing one GUI application, is parsed to extract the exact name, the executable command and the icon. Entries marked as hidden, or lacking a name or an executable, are skipped. The surviving ones are bundled into a small structure and handed to the frontend for rendering and interaction. Figure 5.5 shows that structure.

One detail deserves further analysis in this context. Chromium and Electron-based applications, such as Google Chrome and Visual Studio Code, default to X11 display rendering and require special flags in order to run in the Wayland-only environment of

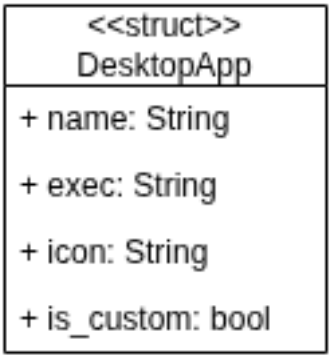


Figure 5.5: The *DesktopApp* structure handed to the frontend for each tile: its display name, the executable command to run, the resolved icon path, and a flag marking whether it is a built-in tile or a discovered application.

the container, forcing them onto the Ozone compatibility platform. To handle this, each parsed executable is checked against a hardcoded set of these identified programs, the Chrome, Chromium, Brave and Edge browsers and the VS Code and VSCodium editors, and where it matches, the appropriate flags (`--enable-features=UseOzonePlatform --ozone-platform=wayland`) are appended to its run command.

For the displayed set to stay current even as applications are installed or removed, the launcher keeps it live through the third-party *notify* crate (a *crate* being Rust's term for a library package), which abstracts a filesystem watcher, an application of the observer, or publish–subscribe, pattern. Once set up, the watcher recursively monitors those same two application directories and emits an event on any change, whether an entry is added, modified or removed. That event, named `apps-changed`, is pushed to the frontend, which reloads its list in response, so that the grid reflects the container's real state without the user ever having to refresh it by hand.

5.5.7. The File Transfer Directory

The environment exposes a single shared folder, `~/Shared`, created inside the container user's home during the user configuration step. It is the one deliberately host-writable point on an otherwise host-read-only container, a property established in Chapter 4, and it serves as the drop zone for moving files in both directions between the host and the container, without involving the network.

Its bidirectional access rests entirely on ownership and ACLs. The directory is owned by the container user, and an *access* ACL opens the *other* class to read and write, which, through the UID translation described in Chapter 4, is exactly the class the real host user falls into. A *default* ACL (`setfacl -d`) is then applied on top, so that every file or subdirectory later created inside `~/Shared` automatically inherits those same permissions. Without it, files dropped in from either side would carry only their original owner's default permissions and could be unreadable from the other. With it, the exchange stays seamless, needing no manual permission fixing each time.

5.5.8. AppImage Environment Cleanup

A subtle but critical detail follows from the launcher itself being shipped as an AppImage. Beyond bundling the dependencies of the application, an AppImage also

bundles standard utilities, such as a Python interpreter, and injects environment variables pointing at those bundled utilities into the running application, and therefore into any child process it spawns. This becomes a problem precisely because most of the container handling happens through carefully constructed terminal commands, arguments and environment variables. The variables the AppImage injects, among them library and plugin search paths such as `LD_LIBRARY_PATH` and the various toolkit paths, would leak into and compromise the ones the launcher sets up for the container.

To prevent this, a *clean_command_env()* function is called before spawning any host process, stripping all of the known variables the AppImage is expected to inject and resetting the data directory path to the host's standard value. While short, this step is critical to the correct functioning of the launcher, since without it the launched host programs would inherit the launcher's own bundled runtime rather than the host's.

5.5.9. The Session Environment File

Rather than scattering environment exports across every launch command, all the variables a container process needs in order to integrate with the host are gathered into a single file per user: `~/.session_env`. It is regenerated on every boot and sourced before attaching a terminal or launching any application, so that each spawned process inherits one consistent, centrally defined session.

The file collects the display variable (`WAYLAND_DISPLAY`), the compliant runtime directory (`XDG_RUNTIME_DIR`, pointing at a dedicated `~/.xdg_run` into which the Wayland socket is symlinked), the host's terminal type (`TERM`, carried over so that command-line applications render correctly), an accessibility-bus silencer (`NO_AT_BRIDGE`) and the Electron Wayland hint (`ELECTRON_OZONE_PLATFORM_HINT`). The message-bus address (`DBUS_SESSION_BUS_ADDRESS`) is appended to the same file slightly later, by the D-Bus setup described next.

5.5.10. The D-Bus Session Bus and Persistent Settings

Most modern desktop applications expect a *D-Bus* session bus to be present. It is the standard inter-process communication channel on Linux, and it also backs *dconf*, the settings-storage backend through which applications save their preferences. Without a session bus, applications still launch, but they flood their output with errors, lose the ability to communicate with one another and, most importantly for the user, cannot persist their configuration across sessions. Since that persistence is one of the project's defining properties, wiring up a working bus inside the container is essential.

The prerequisites for it are installed once, during the user configuration step, following the same distribution family abstraction used elsewhere in the system: the container's distribution is detected and *dbus*, *dconf* and *acl*, along with the distribution-specific compatibility packages, are installed through the matching package manager (`apt`, `dnf`, `pacman` or `zypper`). The container is then given a stable *machine-id*, which D-Bus requires in order to identify the machine: `dbus-uuidgen --ensure` generates one and `/var/lib/dbus/machine-id` is symlinked to `/etc/machine-id`.

The bus itself is managed by a dedicated *initialize_dbus_session()* routine, re-run on every container boot rather than only at first setup. Following the same clean-state robustness principle as the rest of the system, it first kills any leftover `dbus-daemon` owned by the user and removes the stale socket, then launches a fresh daemon at a fixed socket, `~/.xdg_run/bus`, and finally verifies that the bus is actually reachable with a

`dbus-send` probe before reporting success, so that a silently broken session bus is caught immediately.

Finally, the resulting bus address is propagated in the same way as every other session variable: it is appended, as `DBUS_SESSION_BUS_ADDRESS`, to the central `~/.session_env` file described above, which is sourced before attaching the terminal or launching any application. With the bus in place and its address exported, the container's applications gain both their inter-process communication channel and, through *dconf*, the ability to persist their settings from one session to the next.

5.6. The Launcher Frontend

The frontend is the React half of the launcher, the part the student actually sees and interacts with. It owns the entire user interface and connects to the backend only across the Tauri boundary, calling the commands documented above and reacting to the events they emit. It is organized around three ideas: a single command bridge that standardizes how it talks to the backend, a view state machine that drives which screen is shown, and a uniform model for the entries in the application grid. Each is treated in more detail below.

5.6.1. The Command Bridge and the Uniform Result Contract

The frontend and the backend are two separate processes that communicate only across the Tauri boundary, and in two directions. The frontend issues requests to the backend through Tauri's *invoke()* mechanism, a JavaScript call that names a registered Rust command, while the backend pushes asynchronous notifications back through the *emit / listen* event channel. Every command call in the application is funneled through a single module, `api.js`, which exposes one small asynchronous function per backend command, so that no component ever handles a raw command string or an invoke call directly. Concentrating the entire call surface in one place keeps the mechanics of the bridge cleanly abstracted, and provides a single comprehensive list of everything the frontend may ask of the backend.

On top of that, almost every command answers with the same shape: a `(success, errorMessage)` pair. A single wrapper, *runTauriCommand()*, unpacks this pair, surfaces the message to the user when the call has failed, and returns a plain boolean to the caller. This concentrates the error handling in one spot and lets the rest of the interface reason in simple true or false terms. The few commands that return raw data instead of a status pair, the reserved usernames list and the installed applications list, are wrapped separately, each falling back to an empty list on failure, so that a fetch error can never crash the launcher grid.

A second wrapper handles the cases where a failure is not final but a *decision point*. The backend signals these recoverable conditions through agreed-upon sentinel strings rather than free-form messages. A graceful container stop that exceeds its timeout returns `TIMEOUT`, upon which the wrapper asks the user whether to force the stop and, if so, re-issues the same command with the force flag set. On the close path, a `NOT_RUNNING` result is instead interpreted as an already-satisfied success, while a genuine stop failure escalates into a confirmation for an unsafe unmount at the user's own risk. In this way a small, fixed set of sentinels lets the backend hand nuanced control-flow decisions up to

the user, without complicating the otherwise uniform result contract. Listing 5.6 shows the sentinel-based force-stop wrapper.

```
const runTauriCommandWithAlternative = async (commandName, args = {})
    => {
  try {
    const [success, errorMessage] = await invoke(commandName, args);

    if (!success && errorMessage) {
      if (errorMessage === "TIMEOUT") {
        const force = window.confirm("The container is taking too
           long to shut down (an app might be hanging). Would you
           like to force stop it?");
        if (force) {
          const [forceSuccess, forceError] = await invoke(commandName
             , { ...args, force: true });
          if (!forceSuccess && forceError) alert(`Force stop failed:
             ${forceError}`);
          return forceSuccess;
        }
        return false;
      } else {
        alert(`Error: ${errorMessage}`);
      }
    }
    return success;
  } catch (error) {
    alert(`Fatal Application Error: ${error}`);
    return false;
  }
};
```

Listing 5.6: The wrapper that turns the `TIMEOUT` sentinel into a user-driven decision.

5.6.2. The View State Machine

The entire interface is organized as a state machine, driven by two pieces of state held at the top level. The `viewState` selects the single screen currently shown, and the `footerState` selects the persistent footer action, `unmount` while the system is idle or ready, and `stop` while the container is running. The top-level component acts as the orchestrator: it owns the state, performs the backend calls and decides the transitions, while the individual screens are kept almost purely presentational, each receiving only what to display and which callback to invoke. A single mapping function translates the current `viewState` into its corresponding screen component, so that adding or reordering screens is completely isolated from the logic that drives them.

Startup is itself a small pipeline expressed in these same terms. The host, drive and container checks run in sequence, and the first one to fail routes to its matching error screen, `hostError`, `missingEnv` or `corruptEnv`, while passing all three lands on `ready`. From there, each user action, starting, stopping, creating or repairing the environment, configuring the first user, or unmounting, is a handler that chains the relevant backend calls and advances the machine to the next state, falling back to a safe state and surfacing the error whenever a step fails.

Taken together, the machine spans ten states. Besides the four already named, there are the transient `loading`, `startingUp` and `loadingSetupUser` screens shown while

a backend step runs. The `creationFailed` screen is reached when environment creation itself fails. The `setupUser` form is presented on first use, when the container has no personal user yet. Finally there is the `launcher` state, the in-session application grid that is the steady state of the whole application. The complete set of states and transitions is presented in Figure 5.6.

The `launcher` state is best read as a single state rather than as the top-level machine. The interactions available inside it, launching an application, opening a terminal, a file browser or the shared folder, running the runtime health check, or filtering the grid, are local interactions that spawn host processes or update the view in place without ever leaving the state. Representing them as self-transitions within the `launcher` state keeps the top-level machine at one consistent level of abstraction, that of the screens the orchestrator switches between, while still capturing what the user can do once inside.

One transition sits outside the ordinary user actions: closing the window. Rather than letting the window close immediately, the launcher intercepts the close request, holds the window open, and runs the same safe teardown path as an explicit unmount, force-stopping the container if necessary and then unmounting the drive before the application finally exits. This ensures that the drive is never left mounted, nor the container left running, however the user chooses to leave.

5.6.3. Modeling the Launcher Entries

The application grid is assembled from two different kinds of entries that share one uniform tile model but originate at opposite ends of the system. The *discovered* entries are the container's real GUI applications, scraped and parsed on the backend as described earlier and handed to the frontend as structures carrying a name, an executable command and an icon. Because their icons live inside the container's filesystem, each path is converted to a Tauri-safe URL before it can be displayed. The *built-in* entries, the interactive *Terminal*, the *File browser* and the *Shared files* folder, are, by contrast, defined declaratively in the frontend as a short list, each flagged as custom and shipped with a bundled icon asset. The two sets are concatenated and sorted alphabetically together, so that from the user's perspective they form a single coherent grid. Listing 5.7 shows that static built-in list.

```
1 const builtinApps = [
2   { name: 'Terminal',     exec: '__builtin_terminal__', icon:
      terminalIcon,    isCustom: true },
3   { name: 'File browser', exec: '__builtin_explorer__', icon:
      filesIcon,       isCustom: true },
4   { name: 'Shared files', exec: '__builtin_shared__',   icon:
      sharedFilesIcon, isCustom: true }
5 ];
```

Listing 5.7: The built-in entries, defined declaratively and each carrying a reserved sentinel command.

What distinguishes the two kinds at click time is a small dispatch. Each built-in entry carries a reserved sentinel command, `__builtin_terminal__`, `__builtin_explorer__` or `__builtin_shared__`, together with its custom flag, so that a click on a custom tile is routed to the matching dedicated backend command, attaching a terminal, opening the file manager on the home directory, or opening the shared folder, whereas a click on any discovered tile calls the generic application launch command with that entry's

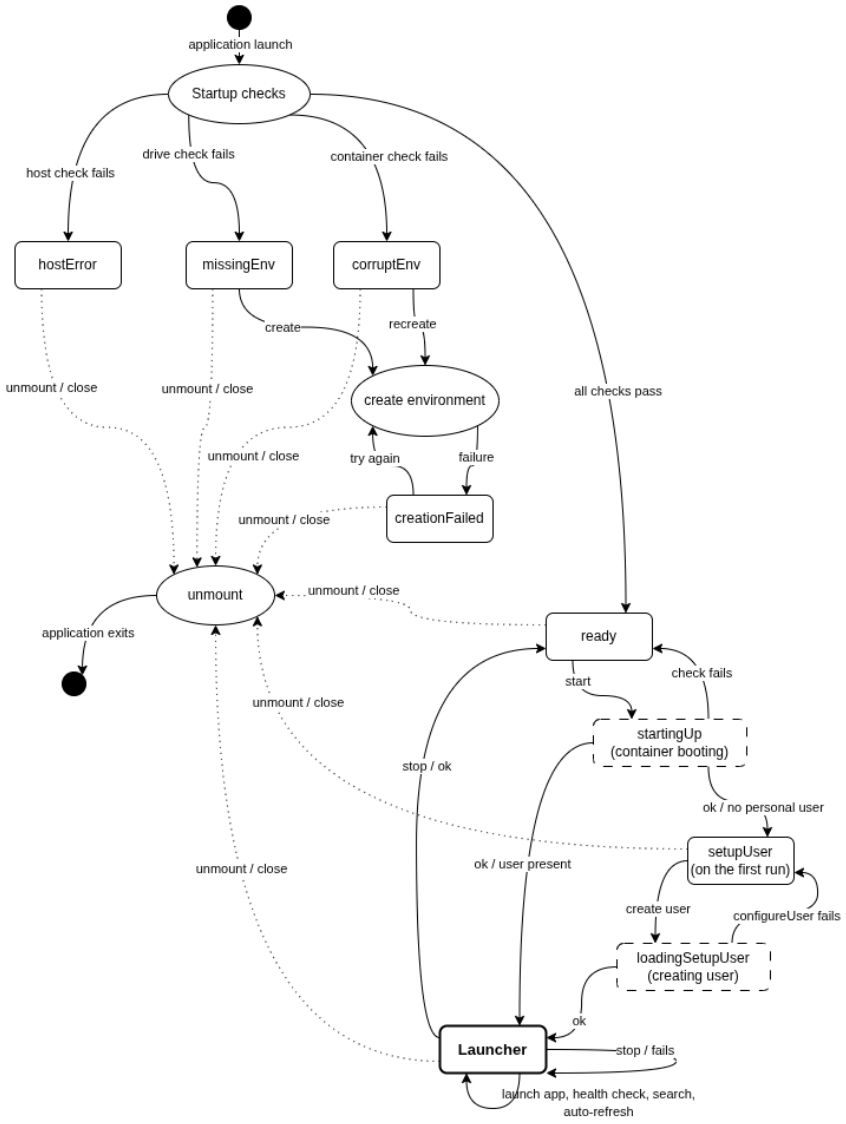


Figure 5.6: The `viewState` state machine: the startup check pipeline routing to the error screens or to `ready`, the provisioning and launch handlers, and the `launcher` steady state with its self-transitions.

executable. This separation makes the custom side easily extensible: introducing a new built-in action amounts to adding one entry to the list and one branch to the dispatch, with no change to the discovery mechanism, while the discovered side grows and shrinks entirely on its own as applications are installed or removed.

The discovered entries additionally expose a right-click context menu, offering to launch the application with a terminal attached, in order to inspect its output, or to copy its executable name. This menu is intentionally suppressed on the built-in tiles, where its options would have no meaning.

Finally, a search field at the top filters the combined grid by name in real time. Connecting to the backend's filesystem watcher, the launcher subscribes to the emitted `apps-changed` event and reloads its list on any change. That reload is debounced by a one-second quiet period, because package managers touch the applications directory multiple times in rapid succession during a single install, and a naive, immediate reaction would make the grid behave chaotically for any such operation.

Chapter 6. Testing and Validation

This chapter evaluates the resulting system, the *testing* step anticipated among the project's objectives, in order to establish whether it actually delivers the properties it was designed around. The reference against which that evaluation is carried out is the set of core properties formulated in Chapter 2: portability across heterogeneous hosts, native execution on the host's own hardware, the running of GUI applications, unprivileged operation, and the isolation and protection of the environment. Each of the validations that follow is a check against one or more of these.

The evaluation proceeds along several complementary directions. It begins with the guarantees the two shipped components verify about themselves: the host configuration tool's ability to confirm its own work simply by being re-executed, and the launcher's automated checks of both the host and the live environment. It then moves to cross-device testing, which proves portability directly across a range of machines and distributions, before turning to the quantitative performance measurements that compare native execution against execution inside the environment. Finally, it addresses the usability of the built-in functionality and the applications the environment is meant to run, and closes with the security validations that put the mechanisms from the security analysis of Chapter 4 to the test.

6.1. Self-Validation Through Re-Execution

The first form of validation is one the host configuration tool performs on itself, and it follows directly from the idempotent design documented in Chapter 5. Every configuration step is written to first inspect the current state of the host and compare it against what the step would produce, reporting the outcome as `JustChecked` when the host already matches and as `ActionCompleted` only when it actually has to change something. Because of this checking-first structure, re-running the tool on a host that is already configured is not only safe, but doubles as a verification of that configuration, since every step re-examines its own artifact against the expected one rather than blindly rewriting it.

Running the tool a second time on the same device therefore demonstrates this exact property. On a correctly configured host, every step in the sequence finds its artifact already present and matching what it should be, and the whole run reports each step as `JustChecked` without altering anything. A complete run of only already configured steps is therefore a direct confirmation that each of the configuration's artifacts, from the networking allowance and the mount units to the GPU rule, the launch stub and the desktop entry, is in place and in the expected state. Figure 6.1 shows this second run reporting the whole configuration as already in place.

The same principle validates the cleanup, this time through the removal mode. After the configuration has been removed once with the `--remove` flag, running the tool

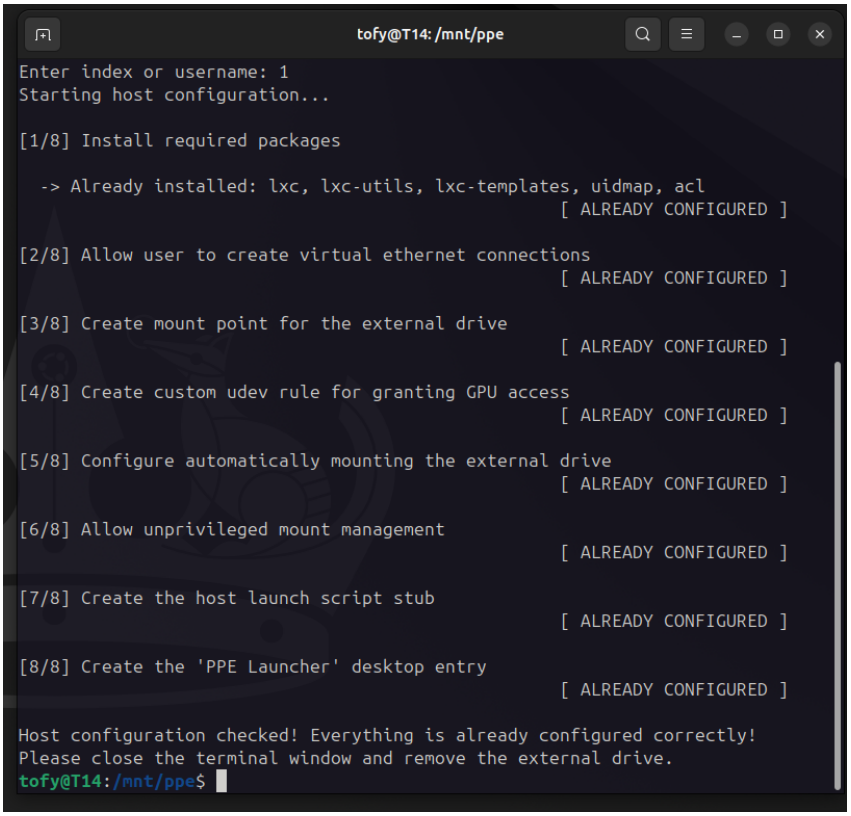


Figure 6.1: The host configuration tool run a second time on an already configured host, reporting every step as already configured rather than performing it.

a second time in the same mode walks the reversal steps again and finds nothing left to undo: each removal step reports its artifact as already absent, so the whole run reports as already clean without further changing the host. This second removal run is what validates the thoroughness of the cleanup, confirming that the first removal left nothing of the configuration behind, and at the same time confirming that the removal is itself idempotent. Figure 6.2 shows this second removal run finding the host already clean.

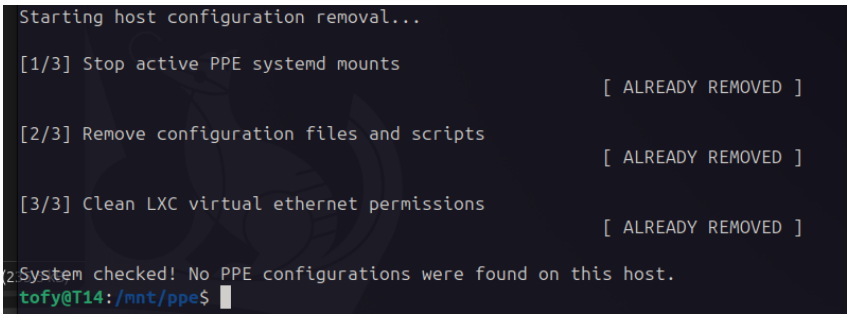


Figure 6.2: The host configuration tool run a second time with the `--remove` flag, reporting every reversal step as having nothing left to undo.

Together, these two re-runs validate the reliability and robustness objective of Chapter 2: the configuration can be applied, verified and reversed without ever leaving the host in an inconsistent state, and the tool is able to report the exact state of its own work on demand.

6.2. Automated Runtime Tests

Beyond the configuration tool checking its own work, the session launcher runs its own automated tests at launch, in two distinct layers. The first is a set of implicit host integrity checks that gate the environment before it is even allowed to boot, and that the user never normally sees. The second is a runtime diagnostics utility, invoked from the launcher menu, that probes the live running environment. The two layers validate different things: that the host is correctly prepared before anything runs, and that the environment's integration with the host is actually functional once it is running.

6.2.1. Implicit Host Integrity Checks Before Boot

As described in Chapter 4 and Chapter 5, the launcher refuses to start the environment until it has confirmed that the host is correctly configured for it. These checks are running silently as part of startup and only surfacing if something is wrong, but they are an important and implicit part of the system's validation, since they verify at runtime the configuration that the host tool put in place. Concretely, before the environment is started the launcher verifies that:

- the required container binaries are present on the host, namely `lxc-start` and `newuidmap`;
- the subordinate UID and GID ranges needed for the identity mapping are allocated;
- the unprivileged networking allowance is in place: the `veth` entry authorizing the container to attach to the host bridge in `/etc/lxc/lxc-usernet`;
- the host's Wayland display socket is present in the user's runtime directory and carries the access-control entry that lets the container's mapped identities read from and write to it;
- the GPU render devices are present under `/dev/dri` and carry the access-control entry granting the container's mapped root identity access to them;
- the container itself exists and looks complete, checking both its configuration and root filesystem and the essential files within it, such as its init program (`/sbin/init`), a shell (`/bin/sh`) and its account database (`/etc/passwd`), together with the traversal permissions on the mount path that let the container be started at all.

When any of these checks fails, the launcher does not proceed to boot the environment but routes instead to an error screen describing the problem. When they all pass, the launch continues silently. Either way, the outcome is the same guarantee, that the host configuration is confirmed to be correct before the environment boots, while making no assumptions about the machine it is running on.

6.2.2. Runtime Integrity Diagnostics

The second layer is the runtime diagnostics utility exposed in the launcher menu. While the startup checks validate the host before boot, this utility validates the live environment once it is running, probing the subsystems that its integration with the host depends on and reporting each. It covers four areas:

- *Network connectivity* — it confirms both that the container has obtained an IPv4 address and that it can actually reach the outside network. External reachability is tested by opening a TCP connection to a well-known public resolver, so that the result reflects real connectivity.

- *GUI integration* — it confirms that the Wayland display socket is present inside the container and that it is both readable and writable as the container user.
- *GPU passthrough* — it confirms that the render devices are present inside the container and accessible, again as the container user, passing when the user can access at least one of the exposed devices, validating hardware acceleration.
- *Container health* — it confirms that the container's own init system reports a healthy state, accepting a running or at worst degraded status as a sign of a functioning system, and adapting to container distributions that do not use it if it is the case.

Run on the working environment, these diagnostics report every area as healthy, as shown in Figure 6.3. Taken together, they confirm that the live container's integration with the host is functional at runtime, and each area maps onto a distinct objective from Chapter 2.

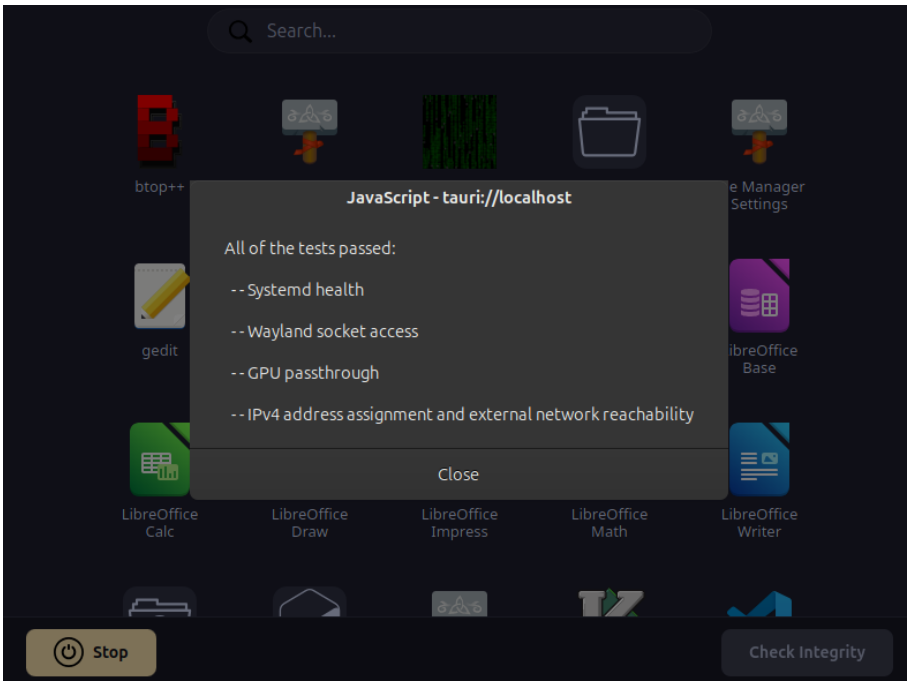


Figure 6.3: The runtime integrity diagnostics invoked from the launcher menu, reporting the network connectivity, GUI integration, GPU passthrough and container health of the live environment.

6.3. Heterogeneous Device Compatibility

Of all the properties formulated in Chapter 2, portability, or host agnosticism, is the one most directly validated by taking a single prepared drive and running it unchanged across a range of different host computers. The devices used for this are more divergent from one to the next, in both hardware and software, so that the same drive is progressively confronted with greater differences between hosts.

The starting point was a Lenovo ThinkPad T480s (Intel Core i5-8350U) running Ubuntu 24.04. On its own, this device establishes the baseline: it proves that the PPE works end to end on a real machine, from host configuration through booting the envi-

ronment to running applications inside it, and it serves as the reference against which the other hosts are compared.

The first genuine test of portability was a second, similar but not identical machine: a Lenovo ThinkPad T14 Gen 1 (Intel Core i5-10210U), also running Ubuntu 24.04. Being close in both hardware and software to the first device, it is the natural first step, checking that the same drive runs unchanged when moved between near-identical hosts. Identical performance on this second device is the first concrete evidence of portability across machines.

The HP ZBook Fury 15 G7 (Intel Core i7-10850H) running Ubuntu 24.04 introduced a considerably larger hardware gap. Its processor is a higher-performance H-series part, a clear step away from the low-power U-series chips of the two laptops, and, more importantly, the machine carries a dedicated NVIDIA Quadro RTX 3000 Mobile GPU alongside its integrated one. This device confirms that only the integrated GPU is passed through to the environment, exactly the behaviour anticipated by the scope boundaries in Chapter 2, which deliberately leave discrete GPUs out of scope. This is perfectly sufficient for the environment's purpose: the graphical applications still render natively and with hardware acceleration on the integrated GPU, so the intended experience is fully preserved on a machine whose discrete GPU the PPE does not touch.

The most divergent host was an Apple MacBook Air (2015 model) running Kubuntu 24.04, which departs from the others in two independent ways at once: unfamiliar Apple hardware and a different desktop environment, KDE rather than GNOME (the Ubuntu desktop environment). Each of these divergences validates a distinct part of the design.

On the hardware side, the host required custom kernel-level drivers to be installed for its networking hardware. Once those drivers were present on the host, network access worked seamlessly inside the container as well, without any further configuration on the environment's side. This is a direct demonstration of one of the PPE's defining advantages: because the container reuses the host's own kernel rather than shipping its own, it inherits the host's kernel-level capabilities and drivers, precisely the reuse that sets this approach apart from the live USB alternative discussed in Chapter 3.

On the software side, the KDE desktop environment proves the system's independence from any single desktop. Its default utilities differ, a different terminal emulator and file manager, yet they are handled transparently by the host tool abstraction described in Chapter 5, which probes for and adapts to whatever the host provides. The desktop environment also integrates several steps differently: the encrypted drive must be unlocked by manually pressing a decryption prompt rather than being offered one automatically as on GNOME, the application launcher interface looks and behaves differently, and even system notifications, such as the confirmation that the drive is safe to remove, are displayed through a different mechanism. In every case the corresponding PPE functionality still works, also since the launcher's desktop entry follows the universal freedesktop.org convention, being found and launched just as easily from KDE's own application launcher as from GNOME's.

This host also surfaced the one performance caveat of the set. Lacking any USB 3 ports, the MacBook Air connects the drive over USB 2 [3], whose lower transfer speed lengthens the initial loading times while leaving the environment perfectly usable in practice. The impact of the connection protocol is measured directly in the performance evaluation later in this chapter.

Taken together, running the same unchanged drive across this deliberately widening spread of machines validates the portability and host agnosticism objective directly,

while surfacing exactly two expected boundaries, the discrete GPU that is out of scope by design and the slower USB 2 connection, both of which the system handles gracefully.

6.4. Performance Validation

The objectives rest on one central promise: that the environment executes natively on the host's own hardware, reusing its kernel rather than emulating one, so that carrying work inside the PPE costs the student nothing in terms of performance. This section sets out to confirm that by running the same workloads directly on the host and then again inside the PPE container on that same host, and showing that the two are indistinguishable in practice.

The measurements were taken on three of the machines introduced earlier: the ThinkPad T480s, the ThinkPad T14 Gen 1 and the HP ZBook. The MacBook Air is left out of this particular comparison, simply because it was no longer available for testing by the time these measurements were carried out. Unless stated otherwise, every test communicates with the drive over its fast USB 3 connection. The single exception is treated on its own in Section 6.4.5, where the ThinkPad T14 repeats the tests over a deliberately slower USB 2 link.

Each benchmark was executed three times in each configuration. To keep the comparison readable, only the averages of those three runs are reported in the tables below, while the complete per-run measurements are collected in Appendix A. The execution time measurement is the one exception to the manual triple run, where the `hyperfine` tool performs its own repeated runs and reports an aggregated mean together with a standard deviation directly, so the figures quoted for it are already averaged over several invocations. In every table the *Difference* column expresses the PPE figure as a percentage change from the native one, so its sign has to be read together with each metric's stated direction, whether higher or lower is the better result.

6.4.1. Processor Performance

The first measurement targets raw processor throughput, the most direct expression of the *native execution* objective. It uses `sysbench`, configured to compute prime numbers up to a fixed bound across all of the machine's logical cores (`sysbench cpu --cpu-max-prime=20000 --threads=$(nproc) run`), and reports the number of such events completed per second, so that a higher figure means a faster result. Because the workload is purely computational and holds its small working set entirely in memory, it measures the processor alone, keeping the drive and its connection out of the timed measurement.

Table 6.1: Processor throughput, native host against PPE (`sysbench`, events per second, higher is better).

Device	Native	PPE	Difference
ThinkPad T480s	2337	2245	−3.9%
ThinkPad T14	2826	2848	+0.8%
HP ZBook	4909	4949	+0.8%

The results, averaged in Table 6.1, show the two columns tracking each other

closely. The largest gap, on the T480s, stays under four percent, and it does not point in a consistent direction: on the other two machines the PPE actually edges slightly ahead of the native run. A genuine virtualization cost would show up as a steady penalty across every device, whereas what is seen here is the ordinary scatter of repeated measurement around a shared value. This is exactly the expected outcome, since a system container shares the host's kernel and schedules its processes on the very same cores, leaving nothing between the computation and the hardware to slow it down.

6.4.2. Graphics Performance and GPU Passthrough

The second measurement examines graphics performance, and with it the GPU passthrough that the *native GPU acceleration* objective depends on. It uses `glmark2` on its Wayland backend, which drives a sequence of OpenGL rendering scenes and condenses them into a single score, higher again being better. On the ZBook, the only machine carrying a discrete GPU, the native run was measured on both its integrated and its dedicated graphics, so that the environment's result can be weighed against each.

Table 6.2: Graphics performance, native host against PPE (`glmark2` score, higher is better).

Device	Native	PPE	Difference
ThinkPad T480s	4266	4200	−1.5%
ThinkPad T14	4901	4931	+0.6%
HP ZBook / with dedicated graphics	6438 / 24365	6168	−4.2% / −74.7%

Here the interpretation carries an extra weight, because the score does more than rank speed: it tells us whether the passthrough worked at all. Had the container failed to reach the host's GPU, it would have silently fallen back to software rendering, and the score would not have only dipped, but collapsed by an order of magnitude. Instead, as Table 6.2 shows, the PPE stays within a few percent of the native result on every machine (on the ZBook of its integrated graphics result) which is only possible if the rendering is in fact running on the host's own GPU. `glmark2` carries a visible run-to-run variation of its own, seen in the raw runs collected in the appendix, and the small deficits observed here, the largest being roughly four percent on the ZBook, sit comfortably inside that variation.

The ZBook's second, much higher figure is where the boundary of the approach becomes visible. Its dedicated NVIDIA card scores nearly four times as high as its integrated one, and the PPE, passed through only the integrated GPU by design, sits well below that, roughly three quarters of the dedicated result. This is the scope limitation discussed earlier in the chapter, now measured: the environment reaches the host's integrated graphics and nothing more. For the graphical applications a student actually runs, editors, browsers and file managers, that is entirely sufficient and their behaviour is unaffected. What stays out of reach is the intense, discrete-GPU-bound work, which the PPE does not set out to serve.

6.4.3. System Call Latency

The third measurement does not look at throughput but at latency, and specifically at the cost of crossing from user space into the kernel, a cost the PPE has a particular

reason to care about. Its *unprivileged operation* rests on UID translation: the container's identities are remapped onto an unprivileged host identity through the user namespace, and that remapping is resolved on the credential path that every system call travels. If it were expensive, it would show here first. The measurement uses `lmbench`'s null system-call latency (`lat_syscall null`), reported in microseconds, where lower is better. In practice the tool repeatedly issues a single, deliberately trivial system call, one that asks the kernel only for the process's own parent identifier, in a loop, and divides the total elapsed time by the number of iterations, so that what is reported is the bare cost of crossing into the kernel and back, stripped of any real work the call might otherwise carry out.

Table 6.3: Null system-call latency, native host against PPE (`lmbench`, microseconds, lower is better).

Device	Native	PPE	Difference
ThinkPad T480s	0.538	0.567	+5.4%
ThinkPad T14	0.097	0.119	+22.7%
HP ZBook	0.081	0.100	+23.5%

Table 6.3 is the one place where a small, consistent difference does appear: the PPE is marginally slower on every machine. This is precisely expected of the UID translation, the extra credential work paid once at each system-call boundary. Its magnitude, however, is negligible in absolute terms, at most around three hundredths of a microsecond, that is, a few tens of nanoseconds per call. The relative figure looks larger on the faster machines, where the baseline itself is only a fraction of a microsecond, but a difference measured in nanoseconds per system call is invisible to any real workload, as the processor and execution time results not showing such penalty confirm. The translation is therefore cheap exactly as the design intended.

6.4.4. Application Launch and Execution Time

Where the previous tests isolate individual subsystems, the fourth measures something closer to lived experience: the time to launch a real application, have it do a unit of work, and exit. It uses `hyperfine`, which runs a command ten times, discards the first three warm-up runs so that the executable is already resident in memory, and reports the mean time together with its standard deviation, in milliseconds (`hyperfine --warmup 3 'google-chrome --headless=new --dump-dom https://example.com'`). The command chosen drives the Google Chrome browser in headless mode to load a page and serialize its contents, testing the container's full launch path, attaching to the container, switching to the user, exporting the session environment and executing the binary from the drive's root filesystem, before any real work even begins.

The averages in Table 6.4 again show no consistent penalty. On two of the three machines the PPE run is in fact the faster of the two. Part of this is explained by the nature of the workload: loading the page involves a network fetch, so a share of the spread reflects fluctuating network conditions rather than anything about the environment itself. The point of the measurement is not that the PPE is faster, which it is not in any real sense, but that a realistic launch task carries no measurable overhead when performed inside it, with the two columns overlapping within their standard deviations.

Table 6.4: Application launch and execution time, native host against PPE (`hyperfine`, milliseconds, lower is better).

Device	Native	PPE	Difference
ThinkPad T480s	587 ± 24	612 ± 6	+4.3%
ThinkPad T14	618 ± 23	599 ± 13	−3.1%
HP ZBook	615 ± 11	525 ± 9	−14.6%

6.4.5. Impact of the Connection Protocol

All of the measurements so far were taken over USB 3. It remains to ask what happens when the same environment is reached over a slower link, both because one of the tested machines, the MacBook Air, only offered USB 2, and because it isolates the one variable, transfer speed, that sets a drive apart from an internal disk. To measure it, the full set of tests was repeated on the ThinkPad T14, this time with a cable that does not support USB 3, forcing the connection down to USB 2 transfer speeds while changing nothing else. In the table below the *Difference* column expresses the USB 2 figure relative to the USB 3 one.

Table 6.5: PPE performance on the ThinkPad T14, over USB 3 against over USB 2.

Test	PPE (USB 3)	PPE (USB 2)	Difference
Processor (events/s)	2848	2835	−0.5%
Graphics (score)	4931	5050	+2.4%
Syscall latency (μs)	0.119	0.116	−2.5%
Execution time (ms)	599 ± 13	538 ± 9	−10.2%

Table 6.5 places the T14's PPE results over the two connections side by side. The differences are unremarkable, each falling within the same run-to-run scatter already seen, with none pointing to a systematic slowdown on the slower link. The execution time figure is even lower over USB 2, again as a result of network variance rather than of the connection. This is the expected result, and it follows directly from how the benchmarks run: once a program has been loaded into memory, it executes entirely from there, and the speed of the link it was originally read from is no longer relevant. `hyperfine`'s warm-up runs make this explicit, loading the executable into the page cache before timing begins, but it holds equally for the compute and latency benchmarks, whose working sets stay resident throughout.

What the connection protocol does bound is not steady performance, but the two things that genuinely depend on moving bytes across the link: the initial load times, felt when the drive is first mounted and when an application is launched for the first time, and the raw read and write throughput to the drive itself. These scale with the protocol's transfer speed [3], so a USB 2 link is felt as slightly longer waits at startup, as already noted for the MacBook Air, rather than as any reduction in how fast work runs once started. This is a property of the storage medium and its connection, not of the PPE, and it would apply in the same way to any system loaded from external storage.

6.5. Usability Validation

Where the previous sections validate that the environment runs and how it performs, this one validates that it is actually convenient and practical to work in, the ease-of-use objective of Chapter 2. It does so in two parts: the built-in functionality the launcher provides, and a representative set of real applications a student would depend on. Every application discussed below was installed from inside the environment through the container distribution's own native package manager (`apt`, for the Ubuntu container the testing was carried out on), which keeps installation both convenient and space-efficient, the latter thanks to the dynamic linking against shared libraries that the native package manager provides.

The three built-in launcher entries, the integrated terminal, the file transfer folder and the host-attached file browser, all worked well across the tested hosts. The integrated terminal behaved perfectly with the host's own terminal emulator on both desktop environments, GNOME Terminal and KDE's Konsole. The host-attached file browser was functional on both as well, opening the container's files in the host's own file manager and displaying their icons correctly on both desktop environments.

On top of these, a set of applications was installed and run, each chosen to check a different aspect of the environment:

- *gedit* — a lightweight graphical text editor, useful for quick edits and note-taking. Beyond simply running, it validates that the container's D-Bus session bus and its *dconf* backend work correctly, since its preferences are successfully saved and restored across sessions. Tests before the D-Bus implementation indeed showed the preference saving mechanism not functional.
- *Visual Studio Code* — a widely used code editor, essential in programming coursework. Being an Electron- and Chromium-based application, it validates the Wayland compatibility layer added for such programs, the display flags without which they would not render in the container's Wayland only environment.
- *Google Chrome* — a web browser, essential for research, documentation and the many web applications a student relies on. As another Chromium-based program it validates the same compatibility layer, and, being a graphics-heavy application, it also tests the native, GPU-accelerated rendering path on the host.
- *htop* — a terminal-based process monitor, exemplifying the command-line tools a student might use. It validates that command-line applications run correctly through the terminal-attached launch path.

Taken together, the built-in integration and this set of real, everyday applications all work through the same seamless launcher, with installation handled by the container's familiar native package manager, which is exactly what the ease-of-use objective expects.

6.6. Security Validation

This final section puts the security mechanisms to a practical test, verifying that the three central protections, drive encryption, UID translation and the read-only from the host property, actually hold on a running system.

The complete drive encryption is the most immediate to confirm. When the drive is inserted into a host, the system prompts for the LUKS password before any of its data can be read, as shown in Figure 6.4, and the host's file manager itself reports the drive

as encrypted, as shown in Figure 6.5. This is the direct confirmation of the stolen drive mitigation from Chapter 4: without the password, the drive reveals nothing.

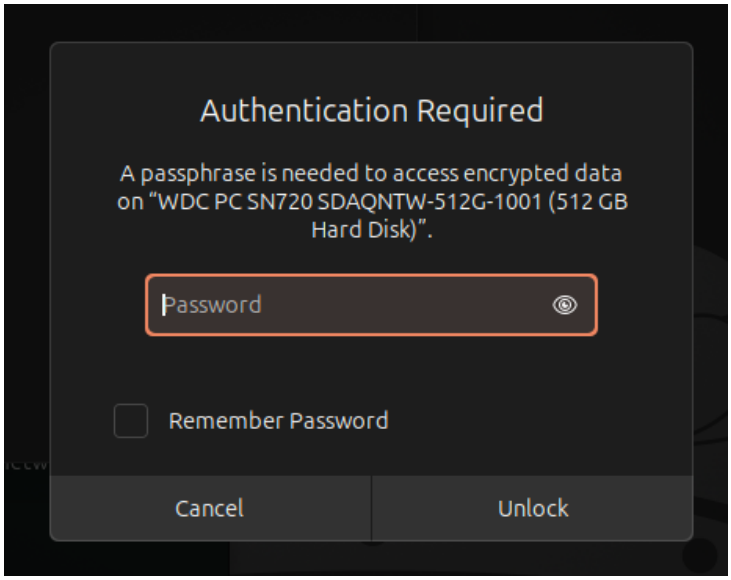


Figure 6.4: The host prompting for the LUKS decryption password when the PPE drive is inserted, before any of its contents can be accessed.

The UID translation is verified by observing the same process from both sides of the boundary at once. With Google Chrome launched from within the container, a terminal attached to the container shows the process owned by the ordinary container user, while a terminal on the host shows that same process owned instead by a distinct, unprivileged mapped identity. A further window shows a terminal of the container's privileged user, one holding sudo rights, from which Chrome is launched, although the browser still runs as an unprivileged process. Figure 6.6 presents these views side by side. Together they demonstrate exactly the property from Chapter 4: the process running inside the container appears on the host under a distinct unprivileged mapped identity, even if it is sudo-capable inside the container, so that authority held inside the container does not carry onto the host itself.

Finally, the read-only host view property is confirmed from the host's own perspective, by listing the container's files while acting as the ordinary host user. As Listing 6.1 shows, none of the container's entries are owned by that host user: the `rootfs` directory is owned by the container's root (`100000`) and the user's files within it by the container's regular user (`101000`), both unprivileged mapped identities that the host user neither matches nor shares a group with. The host user therefore falls into the *other* permission class on every one of them, which, as its permission bits show, grants read and traversal access but never write. A compromised host can thus inspect the environment but cannot tamper with it. The single deliberate exception is the shared transfer folder, appearing as the universally-writable `Shared` directory, which is intentionally left writable from both sides, as established in Chapter 4. The `config` entry listed next to `rootfs` is not part of the container's filesystem but its host-side definition file, and so lies outside this property.

```
tofy@T14:/mnt/ppe/lxc-data/personal-container$ ls -l
total 8
-rw-r-----  1 tofy   tofy    936 May 27 18:35 config
drwxr-xr-x 20 100000 100000 4096 May 27 10:45 rootfs
```

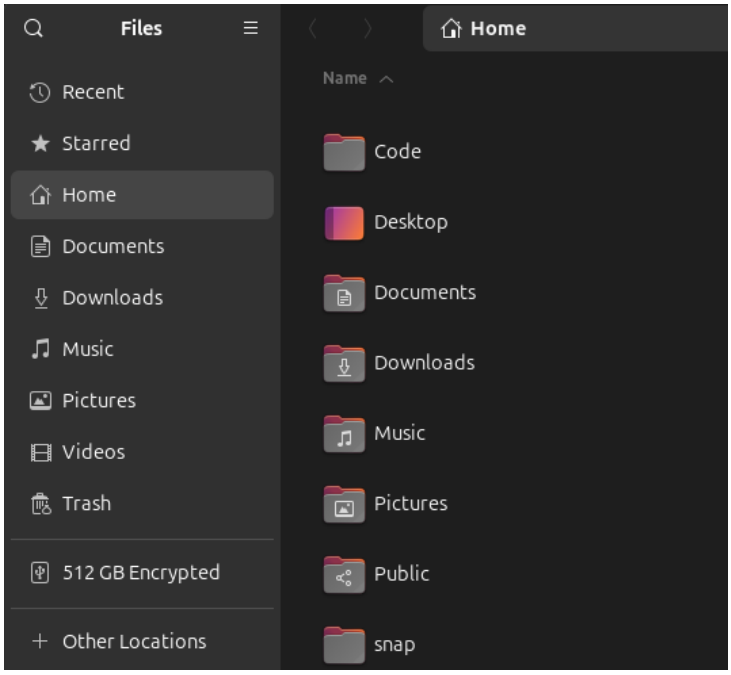


Figure 6.5: The PPE drive reported as encrypted by the host's file manager if the password is not provided.

```
tofy@T14:/mnt/ppe/lxc-data/personal-container$ ls -l rootfs/home/
    vasile/
total 127384
-rw-rw-r--  1 101000 101000 130424340 Jun 11 22:02 google-chrome-
    stable_current_amd64.deb
-rw-rw-r--  1 101000 101000        51 May 28 16:42 laurentiu.py
-rw-rw-r--  1 101000 101000        28 May 27 23:10 myfile.py
drwxrwxrwx+ 2 101000 101000      4096 May 27 18:43 Shared
tofy@T14:/mnt/ppe/lxc-data/personal-container$
```

Listing 6.1: An `ls -l` listing run from the host over the container's directories: every entry under `rootfs` is owned by the mapped container identities (`100000` and `101000`), so the host user falls in the *other* class and has no write permission, the only exception being the universally writable `Shared` folder.

Together, these three checks confirm that the security properties the environment was designed around are not only theoretical, but observably hold on the running system.

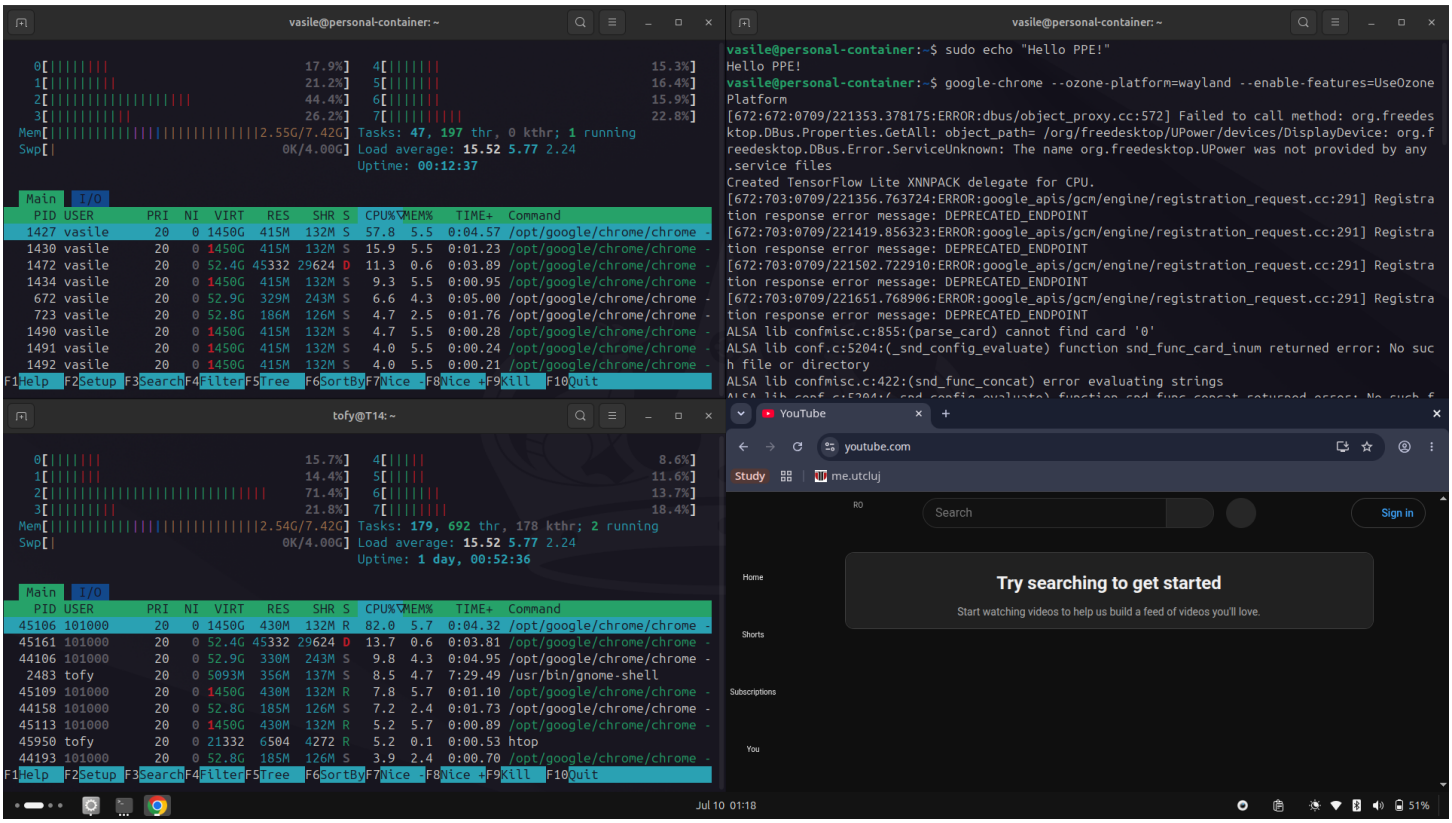


Figure 6.6: The same Google Chrome process seen from both sides of the boundary: owned by the ordinary user in the container's terminal (top left), but by a distinct unprivileged mapped identity in the host's terminal (bottom left), alongside a terminal of the privileged, sudo-capable container user (top right) from which Chrome runs.

Chapter 7. User's Manual

This chapter is a practical guide to preparing and using the PPE. It follows the order in which the system is put into service: the prerequisite resources, an optional step of building the two components from source, the one-time preparation of the external drive, the privileged host configuration, and the everyday use of the environment through the launcher. The first parts address whoever prepares a machine or a drive, most often the classroom administrator, while the last part addresses the student, who simply plugs the drive in and works.

7.1. Prerequisites

The host is any computer running Linux, on one of the currently supported distributions (the Debian/Ubuntu family), with an x86 processor. Beyond the one-time configuration described further below, the host needs nothing else in order to run the environment.

7.2. Building the Two Components

The two components of the system, the configuration tool and the session launcher, are normally distributed as prebuilt executables and simply placed on the drive. Should the person preparing the drive not have these executables at hand, or should they wish to make sure that they have not been tampered with, both can instead be built manually, each from the root directory of its own source code.

The configuration tool is a Rust program, compiled into the static `musl` binary the project relies on. The musl target is added to the toolchain once, with `rustup target add x86_64-unknown-linux-musl`, after which the tool is built with `cargo build --release --target x86_64-unknown-linux-musl`.

The session launcher is a Tauri application whose source is already set to bundle as an AppImage. Its dependencies are first installed with `npm install`, after which it is built with `npm run tauri build`. The executables produced by these two builds are then used in the preparation described next.

7.3. Preparing the Drive

The environment lives on an external drive that has to be prepared once before its first use. As the automatic formatting of the drive is left outside the project's scope, this preparation is done manually and can be carried out either by the administrator or by a regular user preparing their own personal drive.

The drive is formatted as `ext4` and encrypted as a whole with LUKS, at which point the person preparing it chooses the drive's encryption password. From that moment on, every time the drive is inserted it asks for this password before any of its contents can be read, which is what keeps the data confidential on a lost, stolen or otherwise untrusted machine.

The administrator's own drive is an exception: it is used only to carry the configuration tool and to run occasional runtime checks against a host in order to identify and fix problems on it, so it holds no sensitive data and may be left unencrypted.

Finally, the two components of the system, the configuration tool and the session launcher, are copied onto the prepared drive, which is then ready for use.

7.4. Host Configuration

This section addresses the administrator preparing the classroom computers; a regular user may follow the same steps to configure their own personal machine. The configuration tool is the privileged component of the system and is run directly from the drive.

7.4.1. Configuring a Host

The tool must be run with `sudo`. Without it, it warns and stops, since it needs administrative rights to set up the host. On start, it inspects the host and, when the machine is compatible, prompts for the user account the environment will be tied to: it lists the regular users found on the host, and the account is chosen by typing either its index in the list or its username (Figure 7.1).

Once the user is selected, the tool performs its steps and reports the outcome of each one. When it finishes, it instructs to simply unplug the drive and close the terminal.

From this point on the host is configured. When the drive is inserted it is no longer mounted automatically and does not appear in the file manager. It is mounted only on demand, when the "PPE Launcher" desktop entry is started.



Figure 7.1: Selecting the user account to configure the environment for, by index or by username.

7.4.2. Removing the Configuration

To update or undo a configuration, the tool is run again in its removal mode. Because a configured host no longer mounts the drive automatically, the tool stored on the drive is not directly reachable, so the drive has to be mounted through the launcher first:

1. Start the "PPE Launcher" desktop entry, which mounts the drive at `/mnt/ppe/`.
2. From that location, run the configuration tool with `sudo` and the `-remove` flag.
3. As during configuration, select the user whose configuration is to be removed.

7.4.3. Idempotency and Help

Both configuration and removal are idempotent and safe to run a second time: on a repeated run every operation reports `[ ALREADY CONFIGURED ]` in configuration mode, or `[ ALREADY REMOVED ]` in removal mode, which confirms that the first run succeeded.

A help menu is available through `-h` or `-help`. It lists the tool's two modes together with the flags each one requires, and states that the tool must be run with `sudo`. The help option itself can be run without privileges.

7.5. Using the Environment

This section addresses the regular user, the student who plugs an already prepared drive into an already configured host and works inside the environment through the launcher.

7.5.1. Unlocking the Drive and Opening the Launcher

When the drive is inserted, enter the encryption password chosen when the drive was formatted. The environment is then started from the "PPE Launcher" desktop entry, whose exact location depends on the host's desktop environment. On Ubuntu's GNOME, for example, pressing the start button twice brings up the applications list, where the entry can be found.

7.5.2. First Run Configuration

On the first use, two short one-time steps are needed before the environment is ready.

The first is creating the container. If no container exists yet, the launcher says so and offers to create one. Pressing the button opens an interactive terminal that lists the available distributions, versions and architectures (Figure 7.2); the user types the distribution name, its version and the architecture (mandatory `amd64`) from that list.

The second is configuring the in-container user. The first time the container is started, the launcher prompts for the details of the personal account inside the environment: a username and a password, the latter typed twice to make sure it is correct. Some usernames are reserved (for example `root`, `admin` or a distribution's default account); if one of these is entered, the launcher notifies the user and asks for a different one.

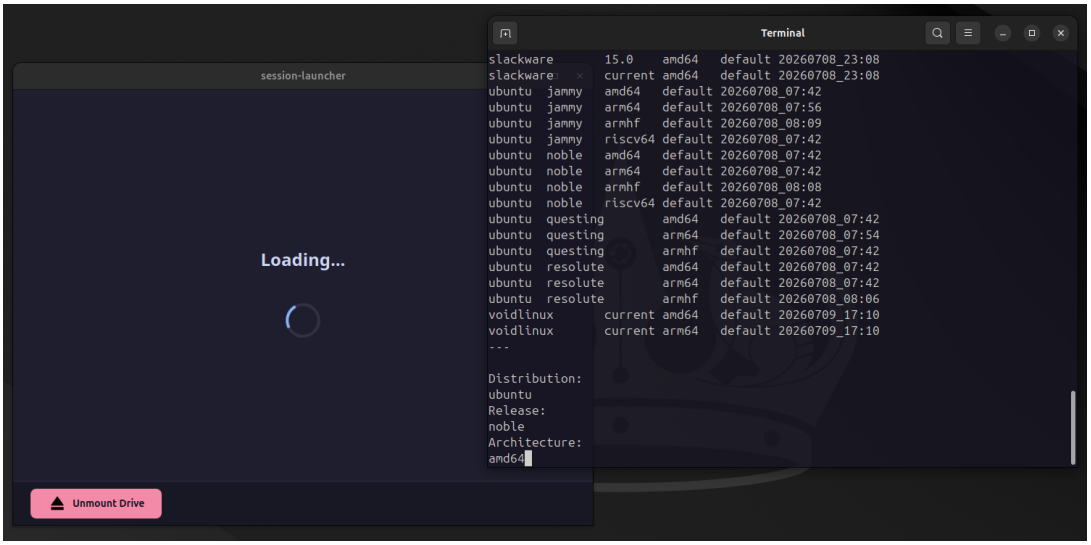


Figure 7.2: The interactive terminal listing the available distributions and versions for the container. (Ubuntu selected as example)

7.5.3. Everyday Use

After the drive is unlocked and the launcher opened, the landing screen offers two actions, *Start* and *Unmount*. Pressing *Start* boots the container (Figure 7.3) and opens the main application page.

The main page presents a grid of all the applications installed in the container, alongside a few built-in entries, a search bar, a stop button and an integrity check button (Figure 7.4). The built-in entries are:

- *Terminal* — opens the container terminal, used to install applications and carry out configurations.
- *Shared files* — opens the `~/Shared/` directory in the host file manager. Files dropped there are accessible for reading and writing from both the host and the container.
- *File browser* — opens the container's home directory (`~/`) in the host file manager, for easy navigation of the environment's files.

A regular application is launched simply by clicking its tile. Right-clicking a tile instead opens a context menu with two options, "Launch with terminal attached" and "Copy executable name". The first is also the way to run command-line applications: these have desktop entries as well, but clicking them does nothing on their own, since they have no graphical window, so they must be started with a terminal attached in order to interact with them.

Typing in the search bar filters the grid by application name. The "Check Integrity" button, on the right of the footer, runs the runtime tests and shows a popup listing the checks that passed and those that failed, together with an error message for each failure.

7.5.4. Shutting Down

There are two ways to shut down, and both must be waited on until they confirm that the drive can be removed.

The cleanest is to press the *Stop* button in the launcher footer to return to the landing screen, then press *Unmount Drive* there, and wait for the notification "PPE



Figure 7.3: The landing screen, with the options to start the container or unmount the drive.

Unmounted" confirming that it is now safe to remove the external drive.

The faster alternative is to simply close the launcher window, which stops the container forcefully and unmounts the drive on its own, again, having to wait for the same notification.

Regardless of the method, the external drive must not be removed before this confirmation appears, so that it is never left in an inconsistent state.



Figure 7.4: The main application page: the grid of installed applications, the built-in entries, the search bar and the footer controls, with an application's right-click context menu open.

Chapter 8. Conclusions

8.1. Summary of Contributions

The Portable Personal Environment set out to close the classroom portability gap described in Chapter 1: the difficulty students face in keeping a secure, personal and fully configured computing environment as they move between the institution's shared computers and their own devices, without being forced to choose between an expensive compact laptop, a heavy one, or the severe limitations of the classroom machines.

The core contribution of this work is a portable, container-based personal environment that answers that gap. Built on an LXC system container and shipped on an encrypted external drive, it runs natively and unprivileged on heterogeneous hosts, reusing each host's own kernel, drivers and hardware while keeping the user's data, applications and configuration entirely on the drive and isolated from the host.

This environment is delivered through two complementary components, both carried on the drive itself. The *host configuration tool* prepares a host once, with administrative privileges, so that the drive can be mounted and run there. The *session launcher* then handles every subsequent session unprivileged, bringing up the container and presenting the interface through which the student runs their applications.

The goal set out at the beginning of the project has been fully achieved, and the resulting solution is fully functional: a student can plug the drive into any pre-configured host and work inside their own secure, personalized environment, exactly as the opening chapters set out to make possible.

8.2. Critical Analysis of the Results

The evaluation carried out in Chapter 6 confirms that the environment meets the objectives set for it. The same drive ran unchanged across the several different machines and desktop environments it was tested on, demonstrating the targeted portability across heterogeneous hosts. The performance measurements showed no considerable difference between running natively on the host and running inside the container, confirming the near-native performance that motivated the choice of a system container. Graphical applications ran with working GUI integration and GPU passthrough on the host's own hardware, and the whole session operated without administrative rights on the host. The security properties analysed in Chapter 4 were likewise put to the test and observed to hold on the running system: the complete drive encryption, the UID translation that keeps in-container authority from reaching the host, and the environment's read-only from the host protection.

At the same time, the evaluation makes the boundaries of the solution just as clear. Support for discrete GPUs was deliberately left outside the scope, due to the incompatible

driver configurations between models, so hardware acceleration is provided on the host's integrated graphics. On a slower USB 2.0 connection, the drive's transfer speed lengthens the initial load times, although, once loaded into memory, programs run natively and the impact largely disappears, leaving the environment perfectly usable in practice. Finally, two of the security threats examined remain only partially resolved: the substitution of the launcher with a malicious copy, for which a signature-based defence is described but not yet implemented, and the unattended decrypted session, which is ultimately a matter of human behaviour rather than something the software can fully prevent.

8.3. Possible Improvements and Further Development

Several directions would extend the system beyond its current state.

The most immediate is broader hardware integration. Audio support would be a natural and low-effort addition, since common Linux sound servers expose their interface as a socket in the user's runtime directory, exactly as the display server does, so the same passthrough mechanism already used for graphics could be extended to sound. In the same direction, further host-side configuration could map other connected devices, such as a camera or additional drives inserted in the host, into the container, so that they become directly accessible from inside the environment.

A second direction concerns ease of use around the drive itself. A dedicated utility that automates the formatting and configuration of a new external drive would remove the one manual preparation step that currently falls outside the project's scope. Complementing this, an option to create backup snapshots of the container would give the user a simple way to preserve and restore the state of their environment.

On the portability side, extending the host configuration tool to distributions beyond the Debian and Ubuntu family would widen the range of hosts a drive can be prepared on. The tool is already organized around a distribution family abstraction for this purpose, so the necessary adaptations concern the distribution-specific commands rather than the structure of the tool.

Finally, the signature-based mitigation against a malicious or compromised launcher, described in the security analysis of Chapter 4, remains to be implemented, closing one of the two residual security gaps identified above.

Taken together, these results show the Portable Personal Environment to be a feasible solution for real educational use: it delivers a secure, portable and personal environment on the classroom hardware that is already there, and the improvements outlined above concern refinement and extension rather than any missing foundation.

Bibliography

[1] Baeldung, "How to Create a Linux Live USB With Persistence Using Easy2boot," https://www.baeldung.com/linux/easy2boot-live-usb-persistence [Accessed 2026.06.28], 2024. [Online]. Available: https://www.baeldung.com/linux/easy2boot-live-usb-persistence

[2] Parallels, "What is a Virtual Desktop Infrastructure (VDI)?" https://www.parallels.com/glossary/virtual-desktop-infrastructure/ [Accessed 2026.06.28], 2026. [Online]. Available: https://www.parallels.com/glossary/virtual-desktop-infrastructure/

[3] Wikipedia contributors, "List of interface bit rates," https://en.wikipedia.org/wiki/List_of_interface_bit_rates#Peripheral [Accessed 2026.07.04], 2026. [Online]. Available: https://en.wikipedia.org/wiki/List_of_interface_bit_rates#Peripheral

[4] L. Bouganim, B. T. Jónsson, and P. Bonnet, "uFLIP: Understanding Flash IO Patterns," in *4th Biennial Conference on Innovative Data Systems Research (CIDR)*, Asilomar, California, USA, 2009. [Online]. Available: https://arxiv.org/abs/0909.1780

[5] University of Washington College of Engineering, "COE-VDI: Virtualized Desktop Infrastructure," https://www.engr.washington.edu/mycoe/computing/studentresources/coevdi.html [Accessed 2026.06.28], 2026. [Online]. Available: https://www.engr.washington.edu/mycoe/computing/studentresources/coevdi.html

[6] Docker, Inc., "Docker 0.9: Introducing Execution Drivers and libcontainer," https://www.docker.com/blog/docker-0-9-introducing-execution-drivers-and-libcontainer/ [Accessed 2026.06.28], 2014. [Online]. Available: https://www.docker.com/blog/docker-0-9-introducing-execution-drivers-and-libcontainer/

[7] Linux Containers, "LXD has been moved to Canonical," https://linuxcontainers.org/lxd/ [Accessed 2026.06.28], 2023. [Online]. Available: https://linuxcontainers.org/lxd/

[8] ——, "Incus: a community fork of LXD," https://linuxcontainers.org/incus/ [Accessed 2026.06.28], 2023. [Online]. Available: https://linuxcontainers.org/incus/

[9] Distrobox, "Distrobox: Use any Linux Distribution Inside Your Terminal," https://distrobox.it/ [Accessed 2026.06.28], 2026. [Online]. Available: https://distrobox.it/

[10] D. Corriveau, "Using Podman, Distrobox, Chezmoi, and Forgejo to Create a Portable and Reproducible Terminal Environment," https://blog.ctms.me/posts/2026-01-21-notes-portable-distrobox-terminal/ [Accessed 2026.06.28], 2026. [Online]. Available: https://blog.ctms.me/posts/2026-01-21-notes-portable-distrobox-terminal/

[11] freedesktop.org, "Desktop Entry Specification, Version 1.5," https://specifications.freedesktop.org/desktop-entry/latest/ [Accessed 2026.07.04], freedesktop.org, 2020. [Online]. Available: https://specifications.freedesktop.org/desktop-entry/latest/

[12] S. Gueron, "Intel Advanced Encryption Standard (AES) New Instructions Set," Intel Corporation, White Paper 323641-001, Rev. 3.0 [Accessed 2026.07.04], 2010. [Online]. Available: https://www.intel.com/content/dam/doc/white-paper/advanced-encryption-standard-new-instructions-set-paper.pdf

[13] Wikipedia contributors, "Bulldozer (microarchitecture)," https://en.wikipedia.org/wiki/Bulldozer_(microarchitecture) [Accessed 2026.07.05], 2026.

Appendix A. Full Performance Measurements

This appendix collects the complete set of measurements behind the averages reported in Section 6.4. For every processor, graphics and system-call benchmark, each of the three runs is listed alongside the mean quoted in the main text. The execution time measurements are not repeated here: because `hyperfine` aggregates its own repeated runs into the mean and standard deviation already given in Table 6.4, that table is itself their complete form.

Table A.1: Processor throughput, all runs (`sysbench`, events per second).

Configuration	Run 1	Run 2	Run 3	Mean
T480s, native	2409	2292	2310	2337
T480s, PPE	2263	2172	2299	2245
T14, native	2846	2809	2824	2826
T14, PPE	2921	2837	2787	2848
T14, PPE (USB 2)	2925	2823	2756	2835
ZBook, native	4915	4923	4890	4909
ZBook, PPE	4935	4979	4932	4949

Table A.2: Graphics performance, all runs (`glmark2` score).

Configuration	Run 1	Run 2	Run 3	Mean
T480s, native	4304	4239	4256	4266
T480s, PPE	4242	4215	4143	4200
T14, native	5215	4656	4832	4901
T14, PPE	5180	4932	4680	4931
T14, PPE (USB 2)	5224	5001	4925	5050
ZBook, native (integrated)	6452	6432	6429	6438
ZBook, native (dedicated)	23917	24366	24812	24365
ZBook, PPE	6216	6151	6138	6168

Table A.3: Null system-call latency, all runs (`lmbench`, microseconds).

Configuration	Run 1	Run 2	Run 3	Mean
T480s, native	0.538	0.534	0.541	0.538
T480s, PPE	0.565	0.566	0.569	0.567
T14, native	0.096	0.100	0.096	0.097
T14, PPE	0.118	0.121	0.117	0.119
T14, PPE (USB 2)	0.115	0.116	0.116	0.116
ZBook, native	0.079	0.082	0.083	0.081
ZBook, PPE	0.101	0.099	0.100	0.100